



UNIVERSITE ABDELMALEK ESSAADI
ECOLE NORMALE SUPERIEURE
Département : Mathématique et informatique
CYCLE LICENCE EN EDUCATION (CLE)
Spécialité : Mathématiques

Fonctionnement de la machine learning supervisée

Réalisé par :

Wejdan Ghailane , Doha Et-touimy , Nada Maghouz

Soutenu le 24 juin 2025 devant le jury composé de :

Pr.Ismail EL hakki	Professeur	ENS Tétouan	Président
Pr.Ismail Tahiri	Professeur	ENS Tétouan	Examineur
Pr.Hayat Benkhalou	Vacataire	ENS Tétouan	Examineur
Pr.El hichami Outman	Professeur	ENS Tétouan	Encadrant

ANNEE UNIVERSITAIRE : 2024-2025

Table des matières

Résumé.	6
Introduction général.	7
1 Régression.	10
1.1 Régression Linéaire	10
1.1.1 Définition de Régression Linéaire	10
1.1.2 - Régression Linéaire simple	11
1.1.3 - Régression Linéaire multiple	13
1.1.4 -Régression Linéaire polynomiale	17
1.2 Descente de Gradient	19
1.2.1 Definition.	20
1.2.2 Compréhension visuelle de Descente de Gradient.	20
1.3 Descente de Gradient Stochastique (SGD)	22
1.3.1 Definition	22
1.3.2 Compréhension visuelle de Descente de Gradient stochastique.	22
1.4 la différence entre Descente de gradient et Descente de gradient stochastique	23
2 Classification	26
2.1 Introduction de la classification	26
2.2 L'algorithme étudiée : SVM	27
2.2.1 La définition de SVM	27
2.2.2 Représentation graphique	28
2.2.3 Les Domaines D'Utilisation	29
2.2.4 Les types de SVM	29
2.2.5 Avantages et inconvénients de SVM	31
3 Étude Mathématique.	32
3.1 Étude Mathématique de la régression linéaire	32
3.1.1 Principe fondamental de la régression linéaire	32
3.1.2 Méthodes des Moindres Carres	33
3.1.3 Fontion de coût et minimisation	35
3.2 Optimisation par Descente de Gradient	37

3.2.1	Principe de la descente de gradient	37
3.2.2	Application à la fonction de coût	38
3.2.3	Application de la Descente de Gradient	38
3.3	Descente de gradient stochastique (SGD)	41
3.3.1	Propriétés des erreurs stochastiques	41
3.3.2	Estimation des paramètres	42
3.3.3	Propriétés des estimateurs	42
3.3.4	Évaluation du modèle	43
3.3.5	Aplication de SGD	44
3.4	Comparaison entre la descente de gradient classique (GD) et la descente de gradient stochastique (SGD)	47
4	Étude mathématique de Classification	50
4.1	Étude Mathématique du SVM Linéaire	50
4.1.1	Objectif de l'algorithme SVM linéaire	50
4.1.2	Hyperplan de séparation	50
4.1.3	Maximisation de la marge	50
4.1.4	Problème d'optimisation primal	51
4.1.5	Formulation duale (multiplicateurs de Lagrange)	51
4.1.6	Fonction de décision	51
4.1.7	Exemple numérique simple	51
5	Technologies et Environnements de Travail	55
5.1	Outils Utilises	55
5.1.1	Python	55
5.1.2	Anaconda	56
5.1.3	Jupyter	57
5.2	Bibliothèques Utilises	58
5.2.1	Matplotlib et NumPy	58
5.2.2	Pandas	60
5.2.3	Scikit-Learn	60
6	Application	62
6.1	Régression	62
6.1.1	Application régression linéaire simple	62
6.1.2	Application de la descente de gradient	64
6.1.3	Descente de Gradient Stochastique (SGD)	67
6.2	Classification	78
6.2.1	Classification d'Emails avec Support Vector Machine (SVM)	78
	Conclusion	84

Table des figures

1.1	Différents types de distribution des données.	11
1.2	Courbe de régression linéaire simple ajustée aux données simulées	13
1.3	Graphique de regression lineaire multiple	16
1.4	Courbe de régression polynomiale ajustée aux données simulées	19
1.5	descente de gradient	21
1.6	descente de gradient stochastique	23
1.7	Comparaison entre la descente de gradient stochastique (à gauche) et la descente de gradient (à droite).	24
2.1	Exemple graphique de classification	26
2.2	Exemple graphique de classification	27
2.3	Caption	30
3.1	representation graphique de l'exemple de moindres carres	35
3.2	Surface de la fonction de coût $J(a, b)$	37
3.3	Descente de gradient	39
3.4	Descente de gradient	40
3.5	Descente de gradient	41
3.6	Visualisation de la fonction de coût $J(\theta)$ et des étapes du SGD	45
3.7	Visualisation de la fonction de coût $J(\theta)$ et des étapes du SGD	46
3.8	Comparaison des trajectoires GD vs SGD sur la fonction de coût	47
3.9	Évolution du paramètre au fil des itérations : GD vs SGD	48
4.1	Courbe de separation svm	53
5.1	Logo de Python	55
5.2	Interface d'Anaconda Navigator	55
5.3	Jupiter	56
5.4	Interface d'Anaconda Navigator	57
5.5	Jupiter	58
5.6	Exemple de visualisation avec Matplotlib	59
5.7	Illustration du fonctionnement de NumPy	59
5.8	Pandas	60
5.9	Scikit learn	61
6.1	Régression linéaire simple)	63
6.2	Régression linéaire obtenue par descente de gradient	66
6.3	Évolution de l'erreur quadratique moyenne (MSE) au fil des époques	66
6.4	Régression linéaire obtenue par descente de gradient stochastique (SGD)	71
6.5	Évolution de l'erreur quadratique moyenne (MSE) au fil des époques	71
6.6	representation graphique	73

6.7	representation graphique	75
6.8	representation graphique	77
6.9	Les Bibliothèques	78
6.10	Chargement et Inspection des Données	79
6.11	Séparation des Caractéristiques (X) et de la Cible (Y)	79
6.12	sortie	80
6.13	Instanciation et Entraînement du Modèle	80
6.14	Instanciation et Entraînement du Modèle sortie	81
6.15	code graphique	81
6.16	representation graphique	82
6.17	Coefficient de Détermination (Score)	82
6.18	Prédictions sur de Nouveaux Exemples	83

Résumé.

Notre projet de fin d'étude explore l'apprentissage supervisé, en se concentrant sur la régression linéaire et la régression stochastique, en analysant leur lien avec les fondements mathématiques et leur application en Python. À travers ce projet, nous cherchons à illustrer l'importance de cette technique dans la résolution de problèmes réels, tout en fournissant une base solide pour des études futures dans le domaine de l'apprentissage automatique. Ce travail met en lumière la puissance de la régression linéaire comme outil d'analyse et de modélisation, et son rôle clé dans le développement de systèmes intelligents. Nous commencerons par une présentation théorique des principes de la régression, expliquant leur rôle dans l'apprentissage supervisé et leur utilisation pour modéliser des relations entre variables. Ensuite, nous étudierons les méthodes de calcul des paramètres..

Enfin, une partie appliquée illustrera l'implémentation de ces algorithmes en Python, à travers des exemples concrets permettant d'évaluer leur efficacité et leurs limites.

Ce projet de fin d'études est une étude complète de l'apprentissage machine supervisé, se concentrant spécifiquement sur les techniques de régression et de classification. Le projet a exploré en détail les concepts de la régression linéaire sous ses diverses formes (simple, multiple, polynomiale) ainsi que les méthodes d'optimisation par descente de gradient (standard et stochastique). Parallèlement, il a abordé la classification à travers l'analyse des Machines à Vecteurs de Support (SVM), en mettant en lumière leurs mécanismes de séparation. L'aspect pratique du projet a été fondamental, avec l'utilisation de l'écosystème Python et de bibliothèques clés telles que NumPy, Matplotlib, et Scikit-learn, notamment pour l'implémentation de modèles de régression pour la prédiction des prix immobiliers. Ce travail a permis de combiner une solide compréhension théorique avec des compétences pratiques en développement et évaluation de modèles prédictifs.

Introduction général .

L'Intelligence Artificielle (IA) est un domaine incontournable dans le paysage technologique actuel, en constante évolution. Elle regroupe un ensemble de techniques et de méthodologies visant à doter les machines de capacités similaires à celles des humains, notamment en matière de perception, de raisonnement, d'apprentissage, de prise de décision et d'interaction avec leur environnement. Le but ultime de l'IA est de créer des systèmes autonomes capables d'effectuer des tâches variées, allant de la simple automatisation de processus à des applications plus complexes comme la reconnaissance vocale, la traduction automatique, la conduite autonome, ou encore la gestion de données massives pour des prédictions dans divers domaines.

L'un des sous-domaines les plus importants et les plus influents de l'IA est l'Apprentissage Automatique (ou Machine Learning, ML), une branche qui s'est particulièrement développée ces dernières années grâce à des avancées majeures dans la disponibilité de données, la puissance de calcul et la sophistication des algorithmes. Le Machine Learning permet aux machines d'apprendre directement à partir des données sans nécessiter une programmation explicite pour chaque tâche spécifique. Au lieu d'être directement codées pour accomplir des tâches, ces machines analysent les données, en détectent les structures, et améliorent leurs performances au fur et à mesure qu'elles en reçoivent de nouvelles. Cela permet d'optimiser des processus, d'améliorer des prédictions ou même de résoudre des problèmes complexes de manière plus efficace que des méthodes traditionnelles basées sur des règles programmées de façon rigide.

L'apprentissage automatique repose sur plusieurs principes fondamentaux. Tout d'abord, les algorithmes de Machine Learning apprennent par itération et ajustement. Cela signifie que, au fil du temps et à mesure qu'ils sont exposés à davantage de données, ces algorithmes ajustent leurs modèles internes pour améliorer leur capacité à faire des prédictions ou à prendre des décisions. Un autre principe majeur est que le processus d'apprentissage est souvent guidé par une évaluation de la performance, mesurée par des critères comme la précision des prédictions ou la réduction de l'erreur. Cet ajustement itératif est essentiel pour permettre aux systèmes d'améliorer leur efficacité, ce qui se traduit par des performances toujours plus élevées dans une variété d'applications.

Le Machine Learning, au-delà de ses principes, se divise en trois grandes catégories qui répondent chacune à des problématiques différentes : l'apprentissage supervisé, l'apprentissage non supervisé et l'apprentissage par renforcement. Chacune de ces catégories repose sur des mécanismes d'apprentissage distincts, mais elles partagent toutes l'objectif

d'exploiter les données pour améliorer les performances des modèles.

L'apprentissage supervisé est sans doute l'approche la plus connue et la plus utilisée dans le Machine Learning. Dans ce cadre, un algorithme apprend à partir d'un ensemble de données étiquetées, c'est-à-dire que chaque entrée de données est accompagnée d'une sortie ou d'une réponse attendue. Par exemple, dans un problème de classification d'images, chaque image est étiquetée avec une catégorie spécifique (par exemple, "chat" ou "chien"). L'algorithme doit apprendre à associer les caractéristiques de l'image à cette étiquette, afin de pouvoir, une fois entraîné, prédire correctement la catégorie d'une nouvelle image. L'apprentissage supervisé est souvent utilisé dans des domaines où des données étiquetées sont disponibles, comme la reconnaissance d'images, la détection de fraude ou encore les diagnostics médicaux.

Dans l'apprentissage supervisé, on distingue principalement deux types de problèmes : la régression et la classification. Les problèmes de régression sont des situations où la variable à prédire est continue. Par exemple, on pourrait vouloir prédire le prix d'une maison en fonction de ses caractéristiques (taille, nombre de chambres, emplacement, etc.). Pour ce type de problème, des algorithmes comme la régression linéaire ou la régression polynomiale sont fréquemment utilisés. D'autre part, les problèmes de classification sont des situations où la variable à prédire est discrète, c'est-à-dire qu'elle prend un nombre limité de valeurs. Un exemple classique est celui de la classification d'e-mails en fonction de leur contenu (par exemple, e-mail "important" ou "non important"). Dans ces cas, des algorithmes comme les arbres de décision, les forêts aléatoires ou les machines à vecteurs de support (SVM) sont utilisés pour diviser les données en classes distinctes.

L'apprentissage non supervisé, quant à lui, diffère de l'apprentissage supervisé en ce sens qu'il ne nécessite pas de données étiquetées. L'objectif ici est de découvrir des structures ou des relations cachées dans les données. L'une des approches les plus courantes dans ce cadre est le clustering, qui consiste à regrouper des données similaires en clusters ou classes. L'algorithme k-means est un exemple classique de méthode de clustering, où l'objectif est de partitionner un ensemble de données en un nombre fixe de groupes (k) de manière à ce que les éléments à l'intérieur de chaque groupe soient aussi similaires que possible. D'autres approches de clustering incluent les méthodes de classification hiérarchique, qui permettent de créer des arborescences de regroupements basées sur les similitudes entre les données.

Outre le clustering, une autre méthode clé en apprentissage non supervisé est la réduction de dimensionnalité, utilisée pour simplifier les données sans en perdre l'information essentielle. Cette technique est particulièrement utile lorsque l'on travaille avec des ensembles de données complexes et volumineux. L'Analyse en Composantes Principales (PCA) et t-SNE sont des méthodes populaires de réduction de dimensionnalité, permettant de projeter des données de haute dimension dans un espace de dimension inférieure, facilitant ainsi leur visualisation et leur analyse.

Enfin, l'apprentissage par renforcement se distingue des deux autres types d'apprentissage par son approche interactive. Dans cet apprentissage, un agent (un programme ou un robot) prend des décisions en fonction de son environnement, dans le but de maximiser

une récompense au fil du temps. L'agent apprend ainsi par essais et erreurs, en ajustant ses actions en fonction des retours reçus sous forme de récompenses ou de pénalités. Cette approche est largement utilisée dans des applications comme les jeux vidéo, la robotique, ou encore dans la gestion de systèmes complexes où l'agent doit naviguer dans un environnement dynamique et prendre des décisions optimales en temps réel. Des algorithmes comme le Q-Learning ou les réseaux de neurones profonds appliqués à l'apprentissage par renforcement (Deep Q-Networks, DQN) ont connu un grand succès dans des domaines tels que les jeux vidéo (par exemple, AlphaGo), ainsi que dans la robotique et la conduite autonome.

Parmi les types de problèmes traités par l'apprentissage supervisé, deux se distinguent : la régression et la classification. Dans ce travail, nous avons choisi d'explorer ces deux dimensions de manière équilibrée, en étudiant d'une part les modèles de régression, et d'autre part les modèles de classification, afin de mieux comprendre leurs mécanismes, leurs applications et leurs performances respectives.

Concernant la régression, nous porterons une attention particulière à la régression linéaire et à ses variantes comme la régression stochastique, en examinant leurs fondements mathématiques, leurs formulations, et leur aptitude à modéliser des relations continues entre variables. Pour ce faire, des outils tels que l'algèbre linéaire et le calcul différentiel seront mobilisés afin d'expliquer comment ces modèles ajustent leurs paramètres pour minimiser l'erreur de prédiction.

En parallèle, nous étudierons la classification supervisée, en nous focalisant sur l'algorithme des machines à vecteurs de support (SVM), un des algorithmes les plus puissants et largement utilisés pour les tâches de classification. Le SVM sera analysé à travers ses bases théoriques, sa capacité à gérer des frontières de décision complexes, ainsi que ses variantes (linéaires et non linéaires). Nous montrerons comment cet algorithme peut être utilisé pour séparer efficacement les classes dans des jeux de données réels, et comment il se compare aux modèles de régression dans des contextes où la variable à prédire est discrète.

Le deuxième chapitre sera dédié à la mise en œuvre pratique de ces deux familles d'algorithmes à l'aide de Python et des bibliothèques spécialisées telles que Scikit-learn. À travers des études de cas concrètes, nous appliquerons la régression linéaire et les SVM sur des ensembles de données réels, en analysant les performances, les limites et les capacités de généralisation de chaque approche. Des comparaisons objectives seront menées afin de mettre en évidence les contextes dans lesquels chaque algorithme excelle.

À travers cette double approche – régression et classification – notre objectif est de fournir une compréhension approfondie des modèles supervisés en Machine Learning, de leurs bases mathématiques et de leur mise en œuvre dans des applications concrètes. Ce travail contribuera également à élargir notre compréhension de domaines applicatifs tels que la prédiction financière, la détection de fraude, le diagnostic médical ou encore l'analyse des tendances de marché, où les décisions automatisées fondées sur des données jouent un rôle clé.

Chapitre 1

Régression.

1.1 Régression Linéaire

1.1.1 Définition de Régression Linéaire

La régression linéaire est une méthode statistique visant à modéliser la relation entre une variable dépendante Y et une ou plusieurs variables indépendantes X . Son objectif est de déterminer la droite qui ajuste au mieux les données, en minimisant l'écart entre les valeurs prédites et les valeurs réelles observées. Cette technique, appelée méthode des moindres carrés ordinaires (MCO), repose sur la minimisation de l'erreur quadratique moyenne (MSE).

Il s'agit de l'une des méthodes les plus fondamentales du Machine Learning, largement utilisée dans des domaines comme l'économie, la finance, la médecine ou encore l'ingénierie. Par exemple, en économie, elle peut servir à étudier l'effet du revenu sur la consommation, tandis qu'en médecine, elle permet d'analyser l'impact de facteurs comme l'âge ou l'IMC sur la pression artérielle.

Dans sa forme simple, la régression linéaire implique une seule variable indépendante, supposée être linéairement liée à la variable dépendante.

representation graphique

Avant d'entreprendre une analyse de régression, il est essentiel de visualiser les données. En régression linéaire simple, on cherche à approximer la relation entre les variables à l'aide d'une fonction f telle que :

$$y_i \approx f(x_i). \quad (1.1)$$

Pour donner un sens à cette approximation, il est nécessaire de définir un critère quantitatif mesurant la qualité de l'ajustement de f aux données.

Ainsi, une étude de régression commence généralement par le tracé du nuage de points (x_i, y_i) , $i = 1, \dots, n$, afin d'évaluer visuellement la pertinence d'un modèle linéaire. Le graphique ci-dessous illustre trois exemples de distributions de données.

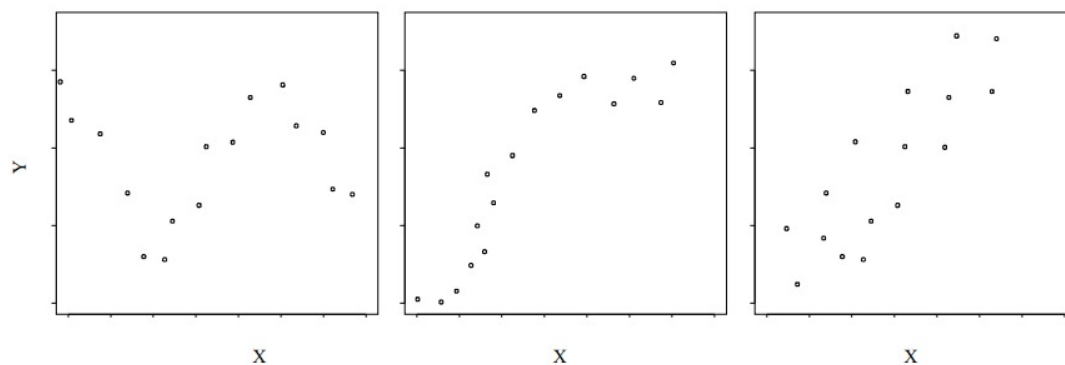


FIGURE 1.1 – Différents types de distribution des données.

Au vu du graphique, il semble inadéquat de proposer une régression linéaire pour les 2 premiers graphiques, le tracé présentant une forme sinusoïdale ou sigmoïdale. Par contre, la modélisation par une droite de la relation entre X et Y pour le dernier graphique semble correspondre à une bonne approximation de la liaison.

Dans la suite de ce chapitre, nous étudierons le cas $f(x) = b_0 + b_1x$.

1.1.2 - Régression Linéaire simple

La régression linéaire simple est une méthode statistique permettant de modéliser la relation entre une variable dépendante Y et une variable explicative X . Elle consiste à ajuster une droite aux données afin de prédire Y en fonction de X . Cette droite est définie par une équation linéaire intégrant un terme d'erreur pour prendre en compte les variations inexpliquées. Une bonne correspondance entre la droite et les points indique que le modèle est pertinent.

L'équation générale de la régression linéaire simple est donnée par :

$$Y = b_0 + b_1X + \varepsilon \quad (1.2)$$

où :

- b_0 est l'ordonnée à l'origine (la valeur de Y lorsque $X = 0$),
- b_1 est le coefficient directeur (la pente de la droite de régression),
- ε représente le terme d'erreur qui regroupe les variations inexpliquées par le modèle.

→ Exemple de la regression lineaire simple

Pour illustrer la régression linéaire simple, nous allons étudier un exemple basé sur une relation entre deux variables : la superficie d'une maison (exprimée en mètres carrés) et son prix (exprimé en milliers de dirhams). L'objectif est de modéliser cette relation à l'aide d'une droite de régression, afin de pouvoir estimer le prix d'une maison en fonction de sa superficie.

Le tableau suivant présente les données que nous allons utiliser :

— **Données d'apprentissage :**

Superficie (m ²)	Prix (en milliers de dirhams)
50	800
60	900
70	1000
80	1100
90	1200

TABLE 1.1 – Données d'exemple : superficie et prix des maisons

On souhaite modéliser la relation entre la superficie d'une maison (variable x) et son prix (variable y) à l'aide d'une régression linéaire simple.

— **Présentation mathématique du modèle :**

$$y = ax + b$$

où :

- $a = 5$: signifie que chaque metre carre supplementaire augmente le prix de 5000 dirhams.
- $b = 650$: represente l'ordonnee a l'origine, c'est-a-dire le prix estime pour superficie nulle (interpretation purement mathematique).

— **Modèle ajusté aux données :**

$$y = 5x + 650$$

Cela signifie que le prix d'une maison augmente en moyenne de 5 kDH (soit 5000 dirhams) pour chaque mètre carré supplémentaire de surface.

— **Exemple de prédiction :**

À partir du modèle de régression linéaire ajusté précédemment :

$$y = 650 + 5x$$

on souhaite estimer le prix d'une maison ayant une superficie de $x = 75 \text{ m}^2$.

Calcul :

$$y = 650 + 5 \times 75$$

$$y = 650 + 375 = 1025$$

Conclusion :

Selon ce modèle, le prix estimé d'une maison de 75 m^2 est d'environ 1 025 000 dirhams (soit 1025 kDH).

— Représentation graphique du modèle :

La figure suivante illustre les résultats obtenus à partir du modèle de régression linéaire ajusté. À partir des données du tableau, elle montre la droite de régression superposée aux points réels représentant les superficies et les prix des maisons. Cette représentation graphique permet de visualiser la tendance linéaire qui relie les deux variables.

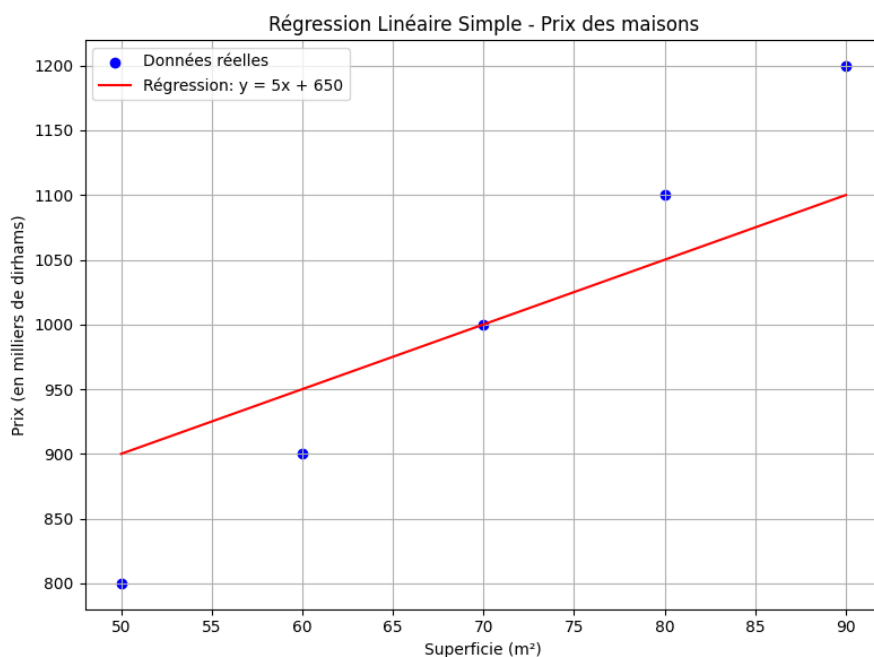


FIGURE 1.2 – Courbe de régression linéaire simple ajustée aux données simulées

— Analyse de la figure :

On observe que la droite (en rouge) suit bien la tendance générale des données (en bleu), capturant la relation croissante entre la superficie et le prix des maisons. Le modèle linéaire simple permet ainsi de modéliser efficacement cette tendance. Toutefois, de légers écarts peuvent apparaître pour certaines valeurs, ce qui laisse envisager l'utilisation éventuelle d'un modèle plus complexe (comme une régression polynomiale) si l'on souhaite affiner la prédiction.

1.1.3 - Régression Linéaire multiple

La régression linéaire multiple est une méthode statistique utilisée pour prédire une valeur en fonction de plusieurs facteurs. Elle permet de mieux comprendre l'influence de différentes variables sur un phénomène donné.

L'équation de la régression linéaire multiple est donnée par :

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon \quad (1.3)$$

où :

- Y est la variable dépendante (réponse),
- $X_1, X_2, \dots, X_p$ sont les variables explicatives (prédicteurs),
- β_0 est l'ordonnée à l'origine (intercept),
- $\beta_1, \beta_2, \dots, \beta_p$ sont les coefficients de régression,
- ε est le terme d'erreur.

L'idée est simple : on cherche à établir une relation entre une variable cible (ce qu'on veut prédire) et plusieurs variables explicatives (les facteurs qui influencent cette cible).

Par exemple, imaginons que l'on souhaite prédire le prix d'une maison. Plusieurs éléments peuvent l'affecter, comme :

- la superficie,
- le nombre de chambres,
- l'emplacement,
- l'âge du bâtiment.

La régression linéaire multiple va analyser ces informations et proposer une relation qui permet d'estimer le prix d'une maison en fonction de ces critères.

Cette méthode est très utile dans de nombreux domaines :

- **Économie** : prévoir la croissance en fonction de plusieurs indicateurs.
- **Marketing** : analyser l'impact de plusieurs campagnes publicitaires.
- **Santé** : comprendre les facteurs influençant une maladie.
- **Science des données** : faire des prédictions basées sur de grandes quantités de données.

Le modèle de régression utilise les données existantes pour apprendre à prédire la valeur cible. Il ajuste un ensemble de coefficients qui indiquent l'importance de chaque facteur. Une fois ce modèle construit, on peut l'utiliser pour estimer de nouvelles valeurs. La régression linéaire multiple est un outil puissant qui permet d'exploiter plusieurs variables pour faire des prédictions et mieux comprendre les liens entre les données.

→ Exemple de la regression lineaire multiple

Voici un exemple illustrant comment on peut utiliser la régression linéaire multiple pour estimer le prix d'une maison en fonction de plusieurs caractéristiques :

— **Données d'apprentissage :**

Surface (m ²)	Nb de chambres	Distance (km) du centre-ville	Année de construction	Prix (MAD)
100	3	2	2015	1 200 000
80	2	5	2010	900 000
120	4	1	2020	1 500 000
90	3	8	2005	850 000
150	5	3	2018	1 800 000

TABLE 1.2 – Exemple de données pour une régression linéaire multiple sur les prix des maisons

— **Présentation mathématique du modèle :**

On cherche à modéliser la relation entre le prix d'un bien immobilier et plusieurs variables explicatives à l'aide d'un modèle de régression linéaire multiple.

$$Y = \theta_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3 + \theta_4 X_4$$

où :

- X_1 : Surface en m²
- X_2 : Nombre de pièces
- X_3 : Distance du centre-ville (km)
- X_4 : Année de construction
- Y : Prix du bien immobilier (en dirhams)
- $\theta_1 = 5000$: chaque mètre carré supplémentaire augmente le prix de 5000 DH.
- $\theta_2 = 100\,000$: chaque pièce supplémentaire augmente le prix de 100 000 DH.
- $\theta_3 = -30\,000$: chaque kilomètre supplémentaire du centre-ville fait baisser le prix de 30 000 DH.
- $\theta_4 = 2000$: chaque année plus récente augmente le prix de 2000 DH.
- $\theta_0 = -2\,500\,000$: constante d'ajustement globale.

— **Modèle ajusté aux données :**

À partir de données simulées ou historiques, on a ajusté un modèle linéaire multiple. Le modèle obtenu est :

$$Y = 5000 \times X_1 + 100\,000 \times X_2 - 30\,000 \times X_3 + 2000 \times X_4 - 2\,500\,000$$

— **Exemple de prédiction**

On souhaite estimer le prix d'un bien immobilier avec les caractéristiques suivantes :

- Surface : 110 m²
- Nombre de pièces : 3
- Distance du centre-ville : 4 km

— Année de construction : 2016

Application de l'équation :

$$\begin{aligned}
 Y &= 5000 \times 110 + 100\,000 \times 3 - 30\,000 \times 4 + 2000 \times 2016 - 2\,500\,000 \\
 &= 550\,000 + 300\,000 - 120\,000 + 4\,032\,000 - 2\,500\,000 \\
 &= \mathbf{2\,262\,000 \text{ MAD}}
 \end{aligned}$$

— Présentation graphique

La figure suivante illustre les résultats obtenus à partir du modèle de régression linéaire multiple ajusté. Elle présente, à partir des données du tableau, la surface de prédiction du prix des maisons en fonction de deux variables explicatives principales : la surface (en m²) et le nombre de chambres. Les points rouges représentent les données réelles observées, tandis que la surface colorée correspond aux valeurs estimées par le modèle. Cette visualisation permet de comparer directement les prix prédits par le modèle avec les prix réels, et d'évaluer visuellement la qualité de l'ajustement.

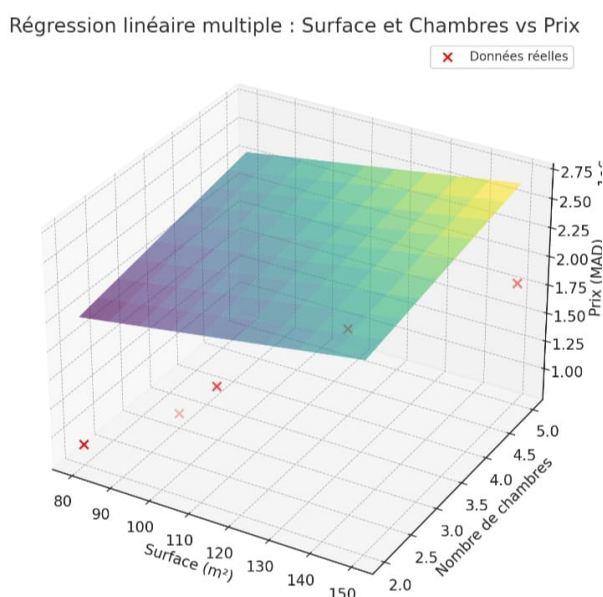


FIGURE 1.3 – Graphe de regression lineaire multiple

— Analyse de la figure

On observe que la surface de régression (en dégradé de couleurs) suit globalement la tendance des données réelles (en rouge), capturant l'effet combiné de la surface et du nombre de chambres sur le prix des maisons. Le modèle linéaire multiple permet de modéliser efficacement la relation croissante entre ces variables et le prix, tout en prenant en compte leur interaction. L'ajustement semble cohérent, surtout dans les zones où les données sont plus concentrées.

1.1.4 -Régression Linéaire polynomiale

La régression polynomiale, en revanche, modélise une relation non linéaire entre les variables dépendantes et indépendantes. Elle étend le concept de régression linéaire en ajoutant des prédicteurs supplémentaires obtenus en élevant chacun des prédicteurs originaux à une puissance.

L'équation de la régression polynomiale est représentée par :

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n + \varepsilon \quad (1.4)$$

où :

- y : Variable dépendante (ou cible), représentant la valeur que l'on cherche à prédire.
- x : Variable indépendante (ou prédicteur), qui influence y .
- b_0 : Ordonnée à l'origine (interception), représentant la valeur de y lorsque $x = 0$.
- $b_1, b_2, \dots, b_n$: Coefficients de régression associés aux puissances de x . Ils déterminent l'influence de chaque terme polynomial sur y .
- $x^2, x^3, \dots, x^n$: Termes polynomiaux permettant de modéliser des relations non linéaires entre x et y .
- ε : Terme d'erreur, représentant la partie de y non expliquée par le modèle. Il prend en compte les variations aléatoires ou d'autres facteurs non inclus dans la régression.

→ Exemple de la regression lineaire polynomiale

Pour illustrer la régression polynomiale, nous allons étudier un exemple basé sur une relation entre deux variables : la superficie d'une maison (exprimée en mètres carrés) et son prix (exprimé en milliers de dirhams).

L'objectif est de modéliser cette relation à l'aide d'une courbe de régression de degré 2, afin de pouvoir estimer le prix d'une maison en fonction de sa superficie, même lorsque cette relation n'est pas strictement linéaire.

— Données d'apprentissage

Le tableau suivant présente les données que nous allons utiliser :

Superficie (m ²)	Prix (en milliers de dirhams)
85.44	32906.64
117.88	45712.67
99.18	38292.75
87.00	32447.88
71.18	28726.11
102.00	38885.51
76.24	30037.47
94.76	36039.52
91.43	34693.94
120.96	46836.44
106.16	40572.27
65.39	26374.40
65.28	26430.10
61.84	25656.11
143.99	55333.87
144.04	56465.86
119.39	44557.19
133.75	50715.10
111.52	41952.37
129.32	50134.49

TABLE 1.3 – Données d'exemple : superficie et prix des maisons

— Présentation mathématique du modèle

On cherche à modéliser la relation entre la superficie d'une maison et son prix à l'aide d'une régression polynomiale de degré 2, afin de représenter une relation non linéaire, comme observé dans les données du tableau.

$$y = \theta_0 + \theta_1 x + \theta_2 x^2$$

- $\theta_2 x^2$: ce terme permet de capturer l'accélération de l'augmentation du prix en fonction de la surface.
- $\theta_1 x$: représente l'effet linéaire de la superficie sur le prix.
- θ_0 : est l'ordonnée à l'origine, c'est-à-dire le prix estimé théorique pour une maison de superficie nulle (interprétation purement mathématique).

— Modèle ajusté aux données

À partir des données du tableau, nous avons ajusté le modèle en utilisant la méthode des moindres carrés. Le modèle final obtenu (par apprentissage sur les données simulées) est par exemple :

$$y = 1234 + 210x + 9x^2$$

— Exemple de prédiction

À partir du modèle de régression polynomiale ajusté précédemment :

$$y = 1234 + 210x + 9x^2$$

on souhaite estimer le prix d'une maison ayant une superficie de 75 m².

Calcul :

$$y = 1234 + 210 \times 75 + 9 \times 75^2$$

$$y = 1234 + 15750 + 5062.5$$

$$y = 67509 \text{ DH}$$

Conclusion : Selon ce modèle, le prix estimé d'une maison de 75 m² est d'environ 67509 dirhams.

— Représentation graphique du modèle

La figure suivante illustre les résultats obtenus à partir du modèle ajusté. Elle montre, à partir des données du tableau, la courbe de régression polynomiale superposée aux points réels représentant les superficies et les prix des maisons.

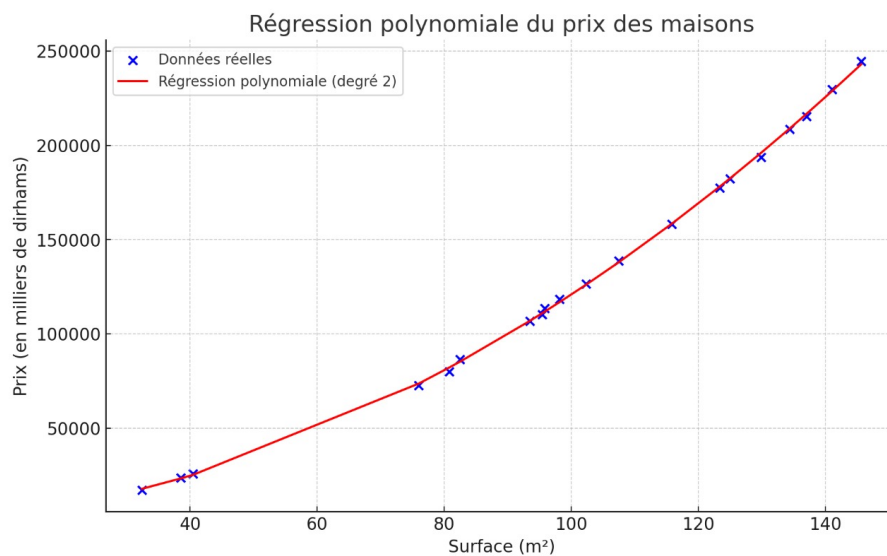


FIGURE 1.4 – Courbe de régression polynomiale ajustée aux données simulées

— Analyse de la figure

On observe que la courbe de régression (en rouge) suit de près la tendance générale des données (points bleus). Cela confirme que le modèle polynômial de degré 2 est bien adapté pour représenter l'évolution non linéaire du prix en fonction de la surface. La précision de l'ajustement est particulièrement visible pour les grandes superficies, où une simple régression linéaire aurait été insuffisante.

1.2 Descente de Gradient

1.2.1 Définition.

La descente de gradient est une méthode d'optimisation itérative utilisée pour minimiser une fonction de coût (ou fonction d'erreur) en ajustant progressivement les paramètres du modèle. Elle consiste à calculer le gradient de la fonction de coût par rapport aux paramètres, puis à déplacer ces paramètres dans la direction opposée au gradient afin de réduire l'erreur.

Formule de base :

$$\theta := \theta - \alpha \cdot \nabla J(\theta)$$

theta : vecteurs des paramètres (poids)

alpha : taux d'apprentissage (learning rate)

$J(\theta)$: fonction de coût

$\nabla J(\theta)$: gradient de la fonction de coût

L'objectif de la méthode de descente de gradient est de trouver un minimum d'une fonction de plusieurs variables le plus rapidement possible. L'idée est très simple, on sait que le vecteur opposé au gradient indique une direction vers des plus petites valeurs de la fonction, il suffit donc de suivre d'un pas cette direction et de recommencer. Cependant, afin d'être encore plus rapide, il est possible d'ajouter plusieurs paramètres qui demandent pas mal d'ingénierie pour être bien choisis.

1.2.2 Compréhension visuelle de Descente de Gradient.

Pour mieux comprendre le fonctionnement de l'algorithme de descente de gradient, on peut l'imaginer comme une goutte d'eau située au sommet d'une montagne. Cette goutte, ne voyant que la pente juste sous elle, suit progressivement la direction de la plus forte descente jusqu'à atteindre la vallée, c'est-à-dire le minimum de la fonction de coût. Ce processus est illustré par le graphe ci-dessous.

Gradient Descent

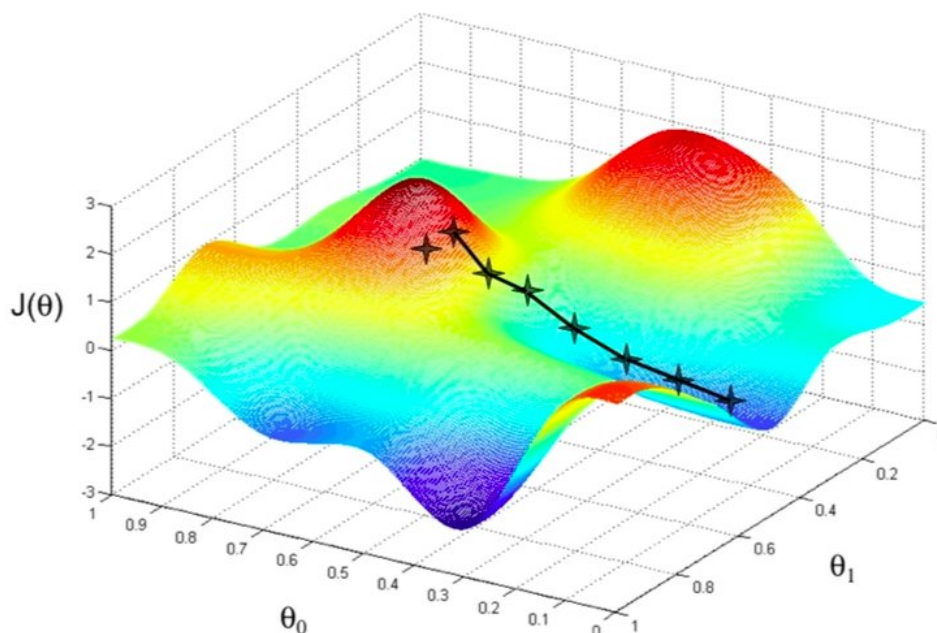


FIGURE 1.5 – descente de gradient

- La surface 3D représente la **fonction de coût** $J(\theta)$, qui mesure l'erreur entre les prédictions et les valeurs réelles.
- Les deux axes horizontaux correspondent aux **paramètres du modèle** : θ_0 et θ_1 .
- L'axe vertical indique la **valeur de la fonction de coût** pour chaque paire (θ_0, θ_1) .
- Les **étoiles noires** symbolisent les différentes **étapes d'itération** de l'algorithme de descente de gradient.
- Le parcours commence depuis une valeur de coût élevée (zone rouge) et descend progressivement vers une valeur minimale (zone bleue).
- À chaque étape, l'algorithme suit la **direction opposée au gradient** pour optimiser les paramètres.
- Le point d'arrivée correspond à un **minimum local ou global** de la fonction de coût.
- Ce processus est similaire au déplacement d'une **goutte d'eau sur une montagne** : elle descend toujours dans la direction où la pente est la plus forte, jusqu'à atteindre une vallée.
- Cette **analogie** aide à comprendre l'idée que l'algorithme "ne voit" que localement la pente sous ses pieds, et ajuste les paramètres petit à petit jusqu'à atteindre le point optimal.

Cette illustration joue un rôle pédagogique essentiel dans la compréhension de l'algorithme de descente de gradient. Elle montre concrètement comment, à partir d'une

position initiale arbitraire (choix aléatoire des paramètres), le modèle évolue en suivant le chemin de la plus forte pente descendante. À chaque itération, le gradient permet de déterminer la direction dans laquelle la fonction de coût diminue le plus rapidement, et les paramètres sont ajustés en conséquence.

Progressivement, ces ajustements rapprochent les paramètres d'un minimum de la fonction de coût, ce qui correspond à une meilleure précision dans les prédictions du modèle. L'ensemble des étoiles noires trace ainsi le parcours de l'apprentissage, à la manière d'une goutte d'eau glissant le long d'une montagne jusqu'à atteindre le point le plus bas.

Ce raisonnement met en lumière un aspect fondamental de l'apprentissage supervisé : l'optimisation itérative. L'objectif n'est pas de trouver immédiatement la solution parfaite, mais de s'en approcher progressivement grâce à une suite d'ajustements bien dirigés. C'est ce processus graduel qui constitue le cœur de nombreux algorithmes en apprentissage automatique.

1.3 Descente de Gradient Stochastique (SGD)

1.3.1 Définition

La descente de gradient stochastique (SGD) est une méthode d'optimisation utilisée pour minimiser une fonction de coût, notamment dans le cas de grands ensembles de données. Contrairement à la descente de gradient classique qui calcule le gradient en utilisant l'ensemble complet des données (batch complet), la SGD met à jour les paramètres du modèle à chaque itération en utilisant une seule donnée (ou un petit échantillon) choisie aléatoirement. Cela permet des calculs plus rapides et une convergence plus rapide, bien que plus bruitée.

L'objectif de la SGD est de minimiser une fonction de coût (ou fonction d'erreur) en ajustant progressivement les paramètres du modèle. En utilisant des mises à jour fréquentes basées sur des exemples individuels (ou petits groupes d'exemples), la SGD permet d'accélérer l'apprentissage, réduire les coûts de calcul et favoriser une convergence plus rapide, surtout lorsque l'on travaille avec des ensembles de données volumineux.

1.3.2 Compréhension visuelle de Descente de Gradient stochastique.

Tout comme la descente de gradient, la descente de gradient stochastique (SGD) cherche à minimiser la fonction de coût. Cependant, au lieu de calculer le gradient à partir de l'ensemble complet des données, elle utilise à chaque itération un seul exemple (ou un petit lot) choisi aléatoirement. Cela rend les mises à jour plus rapides, mais également

plus instables, provoquant des oscillations dans le parcours vers le minimum.

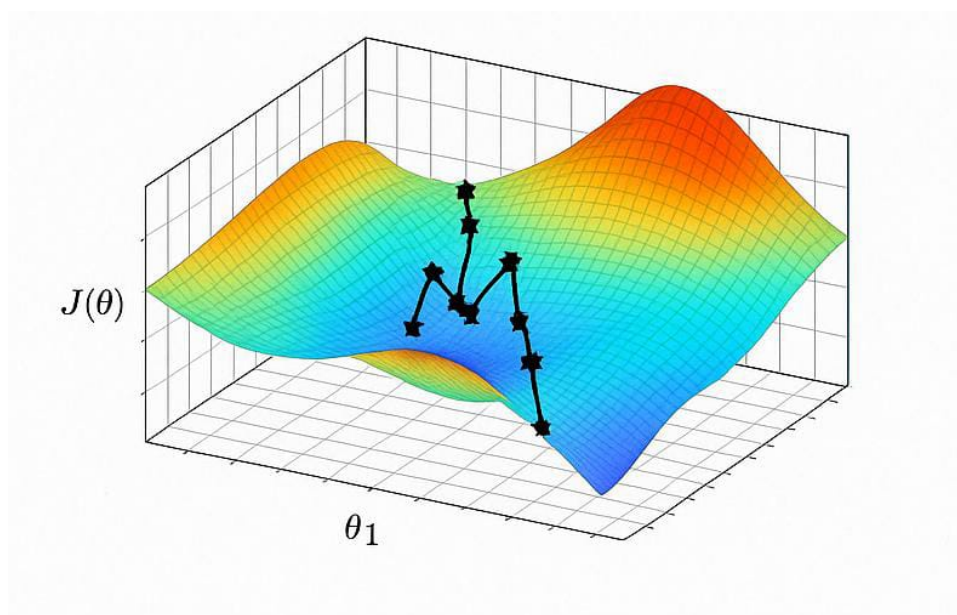


FIGURE 1.6 – descente de gradient stochastique

- La surface 3D correspond à la **fonction de coût** $J(\theta)$, laquelle mesure l'erreur entre les prédictions et les valeurs réelles.
- Les **étoiles noires** indiquent les **modifications successives des paramètres** du modèle à travers les itérations.
- À chaque itération, l'algorithme met à jour les paramètres en utilisant un **exemple aléatoire**, ce qui explique les oscillations du chemin.
- Ce processus continue jusqu'à atteindre un **minimum**, bien que la trajectoire soit moins lisse que celle obtenue avec la descente de gradient classique.

Cette méthode est particulièrement efficace pour les grands ensembles de données, car elle réduit considérablement le temps de calcul. Bien que le chemin parcouru soit plus irrégulier, la descente de gradient stochastique peut converger rapidement vers une solution acceptable, voire optimale, en profitant de la variance introduite par les échantillons aléatoires.

1.4 la différence entre Descente de gradient et Descente de gradient stochastique

L'image ci-dessous présente une comparaison visuelle entre la descente de gradient classique (à droite) et la descente de gradient stochastique (à gauche). Chaque graphe montre l'évolution des paramètres et sur une surface représentant la fonction de coût. Les étoiles noires indiquent les différentes étapes de l'apprentissage.

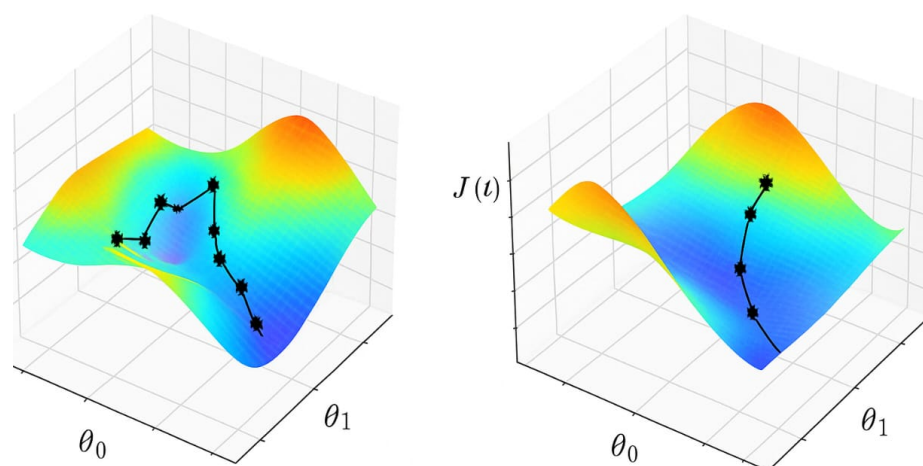


FIGURE 1.7 – Comparaison entre la descente de gradient stochastique (à gauche) et la descente de gradient (à droite).

Le graphe de droite, qui illustre la descente de gradient classique, présente un chemin fluide et direct vers le minimum de la fonction de coût. Les étoiles noires sont alignées de manière régulière, formant une trajectoire stable et continue.

En revanche, le graphe de gauche, représentant la descente de gradient stochastique (SGD), montre une trajectoire instable et irrégulière. Les étoiles noires y sont dispersées, formant un parcours oscillant, avec des déviations fréquentes autour du chemin optimal.

Cette observation visuelle suggère des comportements d'optimisation différents entre les deux approches.

Analyse comparative des deux méthodes :

La descente de gradient classique effectue ses mises à jour à partir de l'ensemble complet des données d'entraînement. Cela lui permet de suivre une direction constante à chaque étape, menant à une trajectoire stable et convergente. C'est une méthode fiable, mais elle devient coûteuse en temps et en mémoire lorsque les jeux de données sont volumineux.

De l'autre côté, la descente de gradient stochastique utilise un seul exemple aléatoire (ou un petit lot) à chaque itération. Cette approche permet des mises à jour rapides et une exécution plus légère, mais elle introduit de la variance dans le calcul du gradient. Cela se traduit par un chemin instable, souvent oscillant, mais qui peut permettre à l'algorithme d'échapper à certains minima locaux.

En résumé, la méthode classique est plus lente mais plus stable, tandis que la méthode stochastique est plus rapide mais plus bruyante. Le choix entre les deux dépend du contexte d'apprentissage, de la taille des données, et des contraintes de performance.

Conclusion :

Critère	Descente de gradient classique	Descente de gradient stochastique (SGD)
Méthode de calcul	Utilise l'ensemble complet des données pour chaque mise à jour.	Utilise un seul échantillon (ou un mini-lot) aléatoire à chaque itération.
Trajectoire observée	Fluide, régulière, stable, converge directement vers le minimum.	Instable, avec des oscillations et un chemin zigzaguant.
Stabilité	Très stable grâce au gradient moyen calculé sur toutes les données.	Moins stable, car influencée par des échantillons aléatoires.
Vitesse d'exécution	Plus lente, surtout avec de grands ensembles de données.	Plus rapide et efficace pour les grands volumes de données.
Précision de la convergence	Plus précise, avec une meilleure stabilité de fin de parcours.	Moins précise, mais peut éviter les minima locaux.
Utilisation recommandée	Jeux de données de taille modérée, où la stabilité est prioritaire.	Très grands jeux de données, ou en phase de pré-entraînement rapide.
Exemple visuel (graphe)	Courbe régulière vers le minimum (graphe à droite).	Trajectoire irrégulière, bruitée (graphe à gauche).

TABLE 1.4 – Comparaison entre la descente de gradient classique et stochastique (SGD)

Chapitre 2

Classification

2.1 Introduction de la classification

La classification est une technique d'apprentissage supervisé qui vise à assigner une ou plusieurs étiquettes (classes) à une variable cible à partir de variables explicatives. L'objectif est de construire un modèle capable de prédire la classe à laquelle appartient une nouvelle observation, en se basant sur un ensemble de données d'entraînement étiquetées. Contrairement à la régression, où la variable à prédire est continue, la classification s'applique aux variables catégorielles. Les algorithmes de classification apprennent à séparer les données en catégories distinctes en identifiant des modèles ou des frontières de décision. Ces algorithmes cherchent à minimiser le taux d'erreur de classification, en utilisant souvent des mesures telles que l'accuracy, la précision, le rappel ou encore la courbe ROC-AUC pour évaluer la performance du modèle. Parmi les algorithmes de classification les plus courants, on trouve :

- K plus proches voisins (K-NN)
- K plus proches voisins (K-NN)
- Arbres de décision (Decision Trees)
- Forêts aléatoires (Random Forests)
- Support Vector Machines (SVM)
- Réseaux de neurones (Neural Networks)
- Naive Bayes

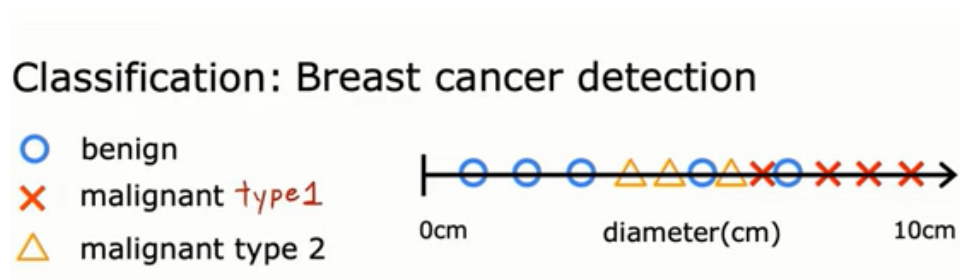


FIGURE 2.1 – Exemple graphique de classification

Ce graphique montre comment les tumeurs sont classées selon leur diamètre :

- bénignes (cercles bleus)
- malignes type 1 (croix rouges)
- malignes type 2 (triangles jaunes)

La classification permet d'identifier le type de tumeur à partir de caractéristiques mesurables.

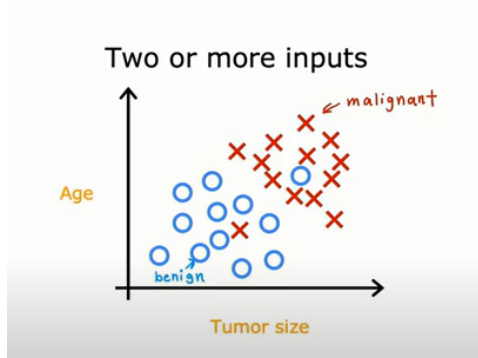


FIGURE 2.2 – Exemple graphique de classification

L'algorithme SVM (Support Vector Machine) est particulièrement reconnu pour sa puissance et sa précision, notamment dans les cas où les données sont linéairement séparables ou presque. Il cherche à construire un hyperplan optimal qui sépare les données en maximisant la marge de séparation, ce qui améliore la capacité du modèle à généraliser sur de nouvelles données.

Une caractéristique importante de SVM est sa capacité à gérer les données non linéairement séparables en utilisant des fonctions noyaux (kernels). Ces fonctions permettent de projeter les données dans un espace de dimension supérieure où une séparation linéaire devient possible.

Dans le cadre de ce projet, nous allons étudier l'algorithme SVM sous ses aspects théoriques et pratiques, en analysant son fonctionnement, son processus d'apprentissage, ainsi que sa représentation géométrique. Nous réaliserons également des expérimentations sur des jeux de données réels, afin d'évaluer ses performances et de le comparer aux algorithmes de régression étudiés précédemment.

2.2 L'algorithme étudiée : SVM

2.2.1 La définition de SVM

Les machines à vecteurs de support (SVM) représentent une méthode d'apprentissage supervisé puissante, principalement utilisée pour les problèmes de classification, bien qu'elles puissent également s'appliquer à la régression (SVR). Leur principe fondamental repose sur la recherche d'un hyperplan optimal dans un espace de dimension N (où N est le nombre de caractéristiques) qui sépare les différentes classes avec la marge maximale possible. Cette marge, définie comme la distance entre l'hyperplan et les points de données

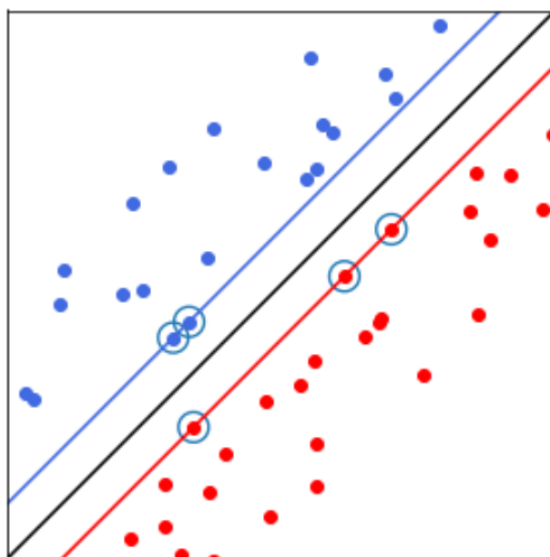
les plus proches (appelés "vecteurs de support"), garantit une meilleure généralisation aux nouvelles données.

L'une des forces majeures des SVM réside dans leur capacité à gérer des problèmes non linéairement séparables grâce à l'astuce du noyau (kernel trick). Cette technique mathématique ingénieuse permet de projeter les données dans un espace de dimension supérieure où une séparation linéaire devient possible, sans pour autant augmenter significativement la charge computationnelle. Les noyaux couramment utilisés incluent le noyau linéaire, polynomial, et RBF (Radial Basis Function), chacun adapté à différents types de distributions de données.

Les SVM offrent plusieurs avantages notables : excellente performance avec des jeux de données de taille modérée, robustesse face au surapprentissage grâce à la maximisation de la marge, et grande flexibilité via les fonctions noyau. Cependant, elles présentent aussi certaines limitations, notamment un coût computationnel élevé pour les grands ensembles de données, une sensibilité au choix des hyperparamètres, et une interprétation moins intuitive que d'autres algorithmes comme les arbres de décision.

2.2.2 Représentation graphique

Comme le montre la figure ci-dessous, leur principe est simple : il ont pour but de séparer les données en classes à l'aide d'une frontière aussi « simple » que possible, de telle façon que la distance entre les différents groupes de données et la frontière qui les sépare soit maximale. Cette distance est aussi appelée « marge » et les SVMs sont ainsi qualifiés de « séparateurs à vaste marge », les « vecteurs de support » étant les données les plus proches de la frontière.



Dans cet espace à deux dimensions, la « frontière » est la droite noire, les « vecteurs de support » sont les points entourés (les plus proches de la frontière) et la « marge » est la distance entre la frontière et les droites bleue et rouge.

2.2.3 Les Domaines D'Utilisation

Comme nous l'avons vu, les SVM dépendent d'algorithmes d'apprentissage supervisé. L'objectif de l'utilisation de SVM est de classer correctement les données non visibles. Les SVM ont de nombreuses applications dans plusieurs domaines.

Certaines applications courantes de SVM sont :

- Détection des visages : SVM classe les parties de l'image en visages et en non-visages et crée une bordure carrée autour du visage.
- Catégorisation du texte et de l'hypertexte : Les SVM permettent la catégorisation du texte et de l'hypertexte pour les modèles inductifs et transductifs. Ils utilisent les données de formation pour classer les documents en différentes catégories. Il catégorise sur la base du score généré, puis compare avec la valeur de seuil.
- Classification des images : L'utilisation de SVM offre une meilleure précision de recherche pour la classification des images. Il offre une meilleure précision par rapport aux techniques de recherche traditionnelles basées sur des requêtes.
- Bio-informatique : Comprend la classification des protéines et la classification du cancer. Nous utilisons SVM pour identifier la classification des gènes, des patients sur la base de gènes et d'autres problèmes biologiques.
- Détection de repli protéique et d'homologie à distance : Appliquez des algorithmes SVM pour la détection d'homologie à distance de protéines.
- Reconnaissance de l'écriture manuscrite : Nous utilisons des SVM pour reconnaître les caractères manuscrits largement utilisés.

2.2.4 Les types de SVM

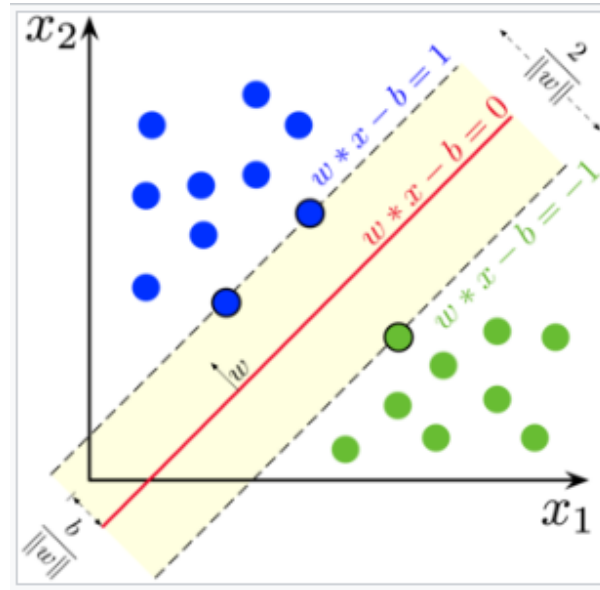
Les Machines à Vecteurs de Support (SVM) peuvent être classées en plusieurs catégories selon leur fonctionnalité et leur approche mathématique. Voici les principaux types

1-SVM Linéaire

Le SVM linéaire fait référence au type de SVM utilisé pour classer des données linéairement séparables. Cela signifie que lorsqu'un ensemble de données peut être divisé en catégories à l'aide d'une seule ligne droite, on parle alors de SVM linéaire, et les données sont dites linéairement séparables. De plus, le classificateur qui effectue cette classification est appelé un classificateur SVM linéaire. Un SVM simple est généralement utilisé

pour résoudre des problèmes de classification et d'analyse de régression.

Le graphique ci-dessus illustre le principe de base d'un SVM linéaire. On observe deux ensembles de données (en bleu et en vert) séparés par une ligne droite appelée hyperplan séparateur (en rouge). L'objectif d'un SVM est de trouver ce plan de séparation de manière à maximiser la marge entre les deux classes. Les lignes en pointillés représentent les marges de part et d'autre de l'hyperplan. Certaines données situées sur ces marges sont appelées vecteurs de support, car elles jouent un rôle clé dans la définition de l'hyperplan optimal.



2-SVM Non Linéaire

Les données non linéaires qui ne peuvent pas être séparées en catégories distinctes à l'aide d'une ligne droite sont classées en utilisant un noyau ou un SVM non linéaire. Ici, le classificateur est appelé classificateur non linéaire. La classification peut être effectuée pour des données non linéaires en ajoutant des caractéristiques dans des dimensions supérieures plutôt que de se limiter à un espace en 2D. Ces nouvelles caractéristiques permettent de définir un hyperplan qui facilite la séparation des classes ou des catégories.

Les SVM avec noyau sont généralement utilisés pour traiter des problèmes d'optimisation comportant plusieurs variables.

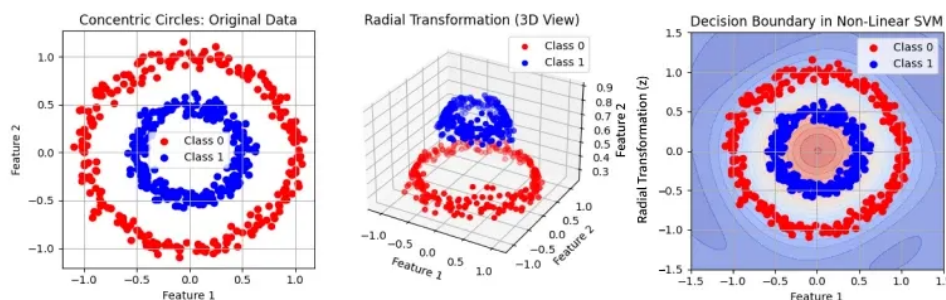


FIGURE 2.3 – Caption

La Figure 2.5 illustre ce processus :

L'image de gauche montre les données initiales sous forme de cercles concentriques (classes 0 et 1), impossibles à séparer avec une frontière linéaire.

L'image centrale présente une transformation radiale des données vers un espace à trois dimensions (grâce à une fonction noyau comme le RBF), où la séparation devient possible.

L'image de droite montre la frontière de décision obtenue après l'application du SVM non linéaire. Cette frontière est ensuite projetée dans l'espace initial, créant une séparation non linéaire efficace entre les deux classes.

Cette technique démontre la puissance des SVM non linéaires pour traiter des problèmes complexes en utilisant des transformations implicites vers des dimensions plus élevées, sans calculer explicitement ces dimensions.

2.2.5 Avantages et inconvénients de SVM

Avantages SVM

- Les SVM sont très bons lorsque nous n'avons aucune idée des données.
- Fonctionne bien avec même des données non structurées et semi-structurées telles que du texte, des images et des arbres.
- L'astuce du noyau est la vraie force de SVM. Avec une fonction noyau appropriée, nous pouvons résoudre n'importe quel problème complexe.

Inconvénients SVM

- Choisir une « bonne » fonction du noyau n'est pas facile.
- Long temps de formation pour les grands ensembles de données

Chapitre 3

Étude Mathématique.

3.1 Étude Mathématique de la régression linéaire

3.1.1 Principe fondamental de la régression linéaire

L'objectif de la régression linéaire est d'établir une relation fonctionnelle entre une variable indépendante et une variable dépendante, en supposant que cette relation peut être approximée par une fonction linéaire de la forme :

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

où :

- Y est la variable dépendante (ou réponse),
- X est la variable indépendante (ou explicative),
- β_0 est l'ordonnée à l'origine (intercept),
- β_1 est le coefficient directeur (pente),
- ε représente l'erreur aléatoire supposée suivre une distribution normale centrée.

Dans le cas d'une régression linéaire multiple avec p variables explicatives, l'équation devient :

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon.$$

- Hypothèses sous-jacentes

Dans un cadre mathématique, la régression linéaire repose sur plusieurs hypothèses fondamentales :

- **Linéarité du modèle :** La relation entre X et Y doit être approximativement linéaire. Si la relation est non linéaire, une transformation des variables peut être nécessaire.

- **Indépendance des erreurs (ϵ)** : Les erreurs doivent être indépendantes les unes des autres, ce qui signifie qu'il ne doit pas y avoir de corrélation entre les résidus d'observation en observation.
- **Homoscédasticité** : La variance des erreurs doit être constante pour toutes les valeurs de X . Si la variance des erreurs change en fonction de X , on parle d'hétéroscédasticité, ce qui peut biaiser l'estimation des coefficients.
- **Normalité des erreurs** : On suppose souvent que les erreurs ϵ suivent une distribution normale $\mathcal{N}(0, \sigma^2)$, ce qui facilite l'inférence statistique sur les coefficients du modèle.

L'objectif principal est donc d'estimer les paramètres et de manière à obtenir la meilleure approximation possible de la relation entre X et Y . Pour cela, il est nécessaire de définir un critère d'ajustement qui permet de quantifier la qualité de l'estimation. En général, ce critère est basé sur la minimisation de l'erreur entre les valeurs prédites et les valeurs observées.

3.1.2 Méthodes des Moindres Carrés

La **méthode des moindres carrés** est une technique analytique utilisée pour estimer les coefficients d'un modèle de régression linéaire en minimisant l'erreur entre les valeurs observées et les valeurs prédites.

Dans un modèle de régression linéaire simple, on cherche à modéliser la relation entre une **variable explicative** x_i et une **variable dépendante** y_i selon l'équation :

$$\hat{y}_i = ax_i + b \quad (3.1)$$

où :

- a est le **coefficient de régression** (pente de la droite),
- b est l'**ordonnée à l'origine** (valeur de y lorsque $x = 0$).

L'**erreur (ou résidu)** est définie comme la différence entre la valeur observée et la valeur prédite :

$$\epsilon_i = y_i - \hat{y}_i \quad (3.2)$$

L'objectif est de trouver les valeurs de a et b qui minimisent la **somme des carrés des erreurs**, définie par :

$$S = \sum_{i=1}^n (y_i - (ax_i + b))^2 \quad (3.3)$$

Minimiser cette somme revient à minimiser l'écart quadratique moyen entre les valeurs observées et prédites, garantissant ainsi le meilleur ajustement possible aux données.

Les valeurs optimales de a et b sont obtenues par dérivation et sont données par les formules suivantes :

$$a = \frac{n \sum (x_i y_i) - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} \quad (3.4)$$

$$b = \frac{\sum y_i - a \sum x_i}{n} \quad (3.5)$$

Ces expressions permettent de calculer directement les coefficients en fonction des données observées, assurant une estimation optimale du modèle.

- Exemple Numérique

Considérons un jeu de données représentant les surfaces (en m²) et les prix (en milliers de dirhams) de cinq maisons :

Surface (m ²)	Prix (kDH)
50	850
60	900
80	1100
100	1500
120	1700

On suppose une relation linéaire entre la surface x et le prix y , donnée par :

$$y = ax + b$$

Pour appliquer la méthode des moindres carrés, nous complétons le tableau avec les colonnes nécessaires :

x	y	x^2	xy
50	850	2500	42500
60	900	3600	54000
80	1100	6400	88000
100	1500	10000	150000
120	1700	14400	204000
$\sum x = 410$	$\sum y = 6050$	$\sum x^2 = 36900$	$\sum xy = 538500$

Le nombre de points est $n = 5$. Les formules des coefficients sont :

$$a = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2} \quad \text{et} \quad b = \frac{\sum y - a \sum x}{n}$$

Substituons les valeurs :

$$a = \frac{5 \times 538500 - 410 \times 6050}{5 \times 36900 - 410^2} = \frac{2692500 - 2480500}{184500 - 168100} = \frac{211000}{16400} \approx 12.87$$

$$b = \frac{6050 - 12.87 \times 410}{5} = \frac{6050 - 5276.7}{5} = \frac{773.3}{5} \approx 154.66$$

L'équation finale de la droite des moindres carrés est donc :

$$y = 12.87x + 154.66$$

Cette équation permet d'estimer le prix d'une maison à partir de sa surface. Par exemple, pour une maison de 90 m^2 :

$$y = 12.87 \times 90 + 154.66 = 1158.3 + 154.66 = 1312.96 \text{ kDH}$$

représentation graphique de l'exemple :

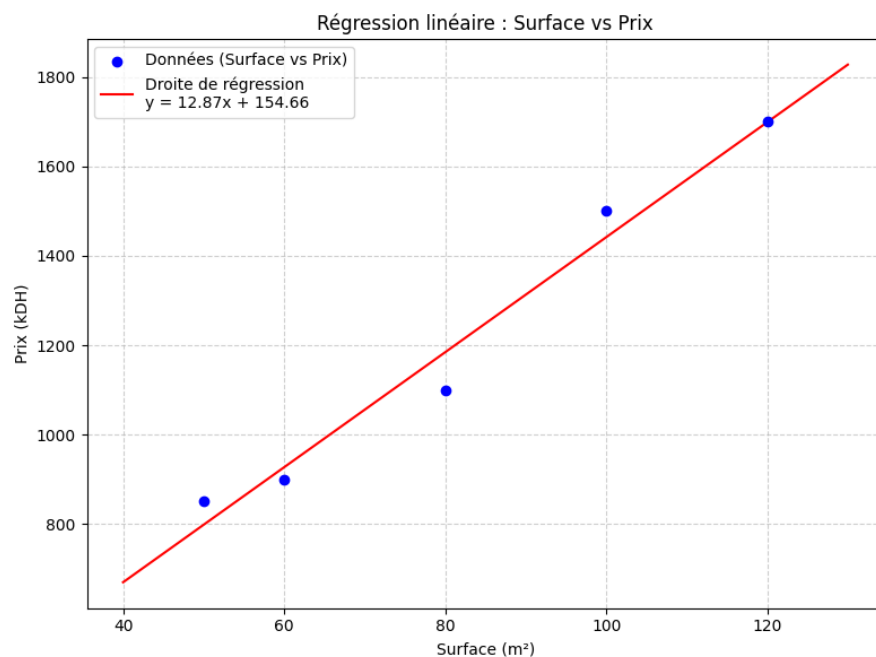


FIGURE 3.1 – représentation graphique de l'exemple de moindres carrés

Ce graphique montre la relation entre la surface d'une maison (en m^2) et son prix (en milliers de dirhams) grâce à la méthode des moindres carrés. Les points bleus représentent les données observées, tandis que la droite rouge, $y = 12.87x + 154.66$, est la droite de régression qui modélise cette relation. Elle indique qu'un mètre carré supplémentaire augmente le prix d'environ 12.87 kDH. Ce graphique permet donc d'estimer le prix d'une maison selon sa surface.

3.1.3 Fonction de coût et minimisation

La **fonction de coût** $J(a, b)$ permet de mesurer l'écart entre les valeurs prédites par un modèle linéaire $h(x) = ax + b$ et les valeurs réelles observées y . Elle est utilisée pour

évaluer la performance de l'ajustement du modèle aux données :

$$J(a, b) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2 = \frac{1}{2n} \sum_{i=1}^n (ax^{(i)} + b - y^{(i)})^2$$

où :

- n est le nombre de points de données (ici $n = 5$),
- $x^{(i)}$ représente la surface de la $i^{\text{ème}}$ maison,
- $y^{(i)}$ est le prix réel correspondant,
- $h(x^{(i)})$ est la valeur prédite par le modèle.

-Application à l'exemple :

À partir de l'équation de la droite obtenue par la méthode des moindres carrés :

$$y = 12,87x + 154,66$$

Nous complétons le tableau suivant :

Surface x (m ²)	Prix réel y (kDH)	Prédiction $h(x)$	$(h(x) - y)^2$
50	850	798,16	2693,46
60	900	926,86	723,34
80	1100	1184,26	7097,06
100	1500	1441,66	3405,86
120	1700	1699,06	0,88

On calcule alors la fonction de coût :

$$J(a, b) = \frac{1}{2 \times 5} (2693,46 + 723,34 + 7097,06 + 3405,86 + 0,88) = \frac{1}{10} \times 13920,6 = 1392,06$$

La valeur $J = 1392,06$ représente l'erreur quadratique moyenne entre les prix prédits et les prix réels pour les cinq maisons considérées. Cette valeur relativement faible montre que la droite de régression obtenue modélise correctement la relation entre la surface et le prix. Cela confirme la pertinence de l'approche des moindres carrés dans ce contexte.

-Visualisation géométrique de la fonction de coût

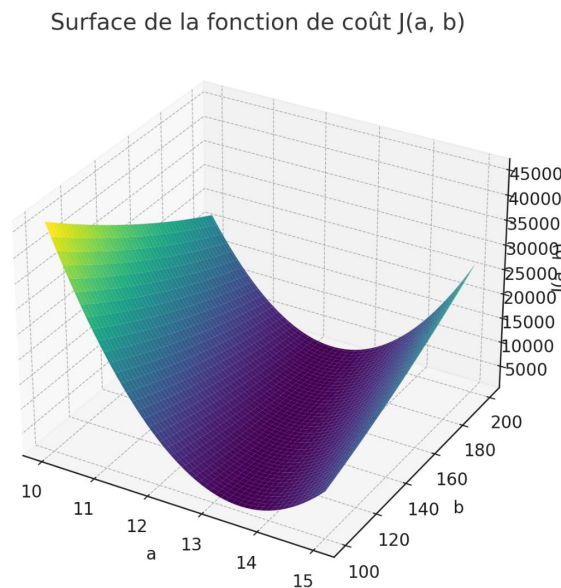


FIGURE 3.2 – Surface de la fonction de coût $J(a, b)$

Cette figure représente la surface de la fonction de coût $J(a, b)$ utilisée dans la méthode des moindres carrés. Elle est calculée à partir des cinq données représentant la relation entre la surface des maisons et leur prix. Chaque point sur cette surface correspond à une combinaison de paramètres (a, b) et à l'erreur quadratique associée. Le point le plus bas sur cette surface représente le couple $(a = 12.87, b = 154.66)$, qui minimise l'erreur, et constitue donc la solution optimale pour la droite de régression.

3.2 Optimisation par Descente de Gradient

3.2.1 Principe de la descente de gradient

La descente de gradient est une méthode itérative utilisée pour minimiser une fonction en suivant la direction de la pente (le gradient) la plus forte en descente. L'idée principale est de mettre à jour les paramètres du modèle de manière à réduire progressivement l'erreur de prédiction. À chaque itération, les paramètres sont ajustés selon la règle suivante :

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)})$$

où :

- θ représente le vecteur des paramètres (dans notre cas, a et b),
- α est le taux d'apprentissage (*learning rate*),
- $\nabla J(\theta)$ est le gradient de la fonction de coût.

3.2.2 Application à la fonction de coût

Dans le cas de la régression linéaire simple avec la fonction de coût suivante :

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m (y_i - (ax_i + b))^2$$

Le gradient s'écrit comme :

$$\frac{\partial J}{\partial a} = -\frac{1}{m} \sum_{i=1}^m x_i \cdot (y_i - (ax_i + b))$$

$$\frac{\partial J}{\partial b} = -\frac{1}{m} \sum_{i=1}^m (y_i - (ax_i + b))$$

Les paramètres sont donc mis à jour à chaque itération selon :

$$a \leftarrow a - \alpha \cdot \frac{\partial J}{\partial a}, \quad b \leftarrow b - \alpha \cdot \frac{\partial J}{\partial b}$$

3.2.3 Application de la Descente de Gradient

À partir du même jeu de données, nous allons estimer les paramètres a et b par la méthode de descente de gradient classique.

Fonction de coût à minimiser :

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m (ax_i + b - y_i)^2 \quad \text{avec } m = 5$$

Règles de mise à jour :

$$\begin{cases} a := a - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (ax_i + b - y_i) \cdot x_i \\ b := b - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (ax_i + b - y_i) \end{cases}$$

Initialisation des paramètres :

$$a = 0, \quad b = 0, \quad \alpha = 0,0001$$

Première itération :

x_i	y_i	$(ax_i + b - y_i)$
50	850	-850
60	900	-900
80	1100	-1100
100	1500	-1500
120	1700	-1700

$$\frac{\partial J}{\partial a} = \frac{1}{5} [(-850) \cdot 50 + (-900) \cdot 60 + (-1100) \cdot 80 + (-1500) \cdot 100 + (-1700) \cdot 120] = \frac{-535500}{5} = -107100$$

$$\frac{\partial J}{\partial b} = \frac{1}{5} (-850 - 900 - 1100 - 1500 - 1700) = \frac{-6050}{5} = -1210$$

Mise à jour des paramètres :

$$a := 0 + 0,0001 \cdot 107100 = 10,71 \quad , \quad b := 0 + 0,0001 \cdot 1210 = 0,121$$

Ainsi, après une première itération, les paramètres estimés deviennent :

$$a^{(1)} = 10,71 \quad , \quad b^{(1)} = 0,121$$

En répétant ce processus plusieurs fois, les paramètres convergeront vers des valeurs proches de celles obtenues par la méthode des moindres carrés :

$$a \approx 12,87, \quad b \approx 154,66$$

Remarque : La vitesse de convergence dépend fortement du taux d'apprentissage α . Un α trop élevé risque de provoquer des oscillations ou une divergence ; un α trop faible ralentit la convergence.

Illustration graphique suggérée :

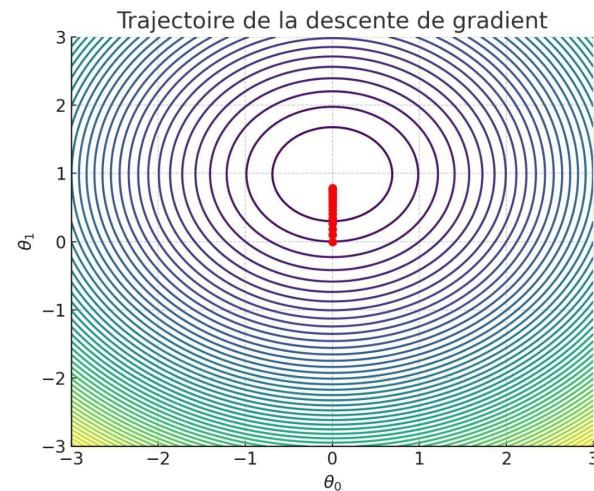


FIGURE 3.3 – Descente de gradient

La figure suivante représente la trajectoire empruntée par l'algorithme de descente de gradient classique sur les courbes de niveau de la fonction de coût $J(\theta_0, \theta_1)$.

Cette visualisation est construite à partir du même jeu de données que précédemment, où l'on modélise la relation linéaire entre la surface d'un logement et son prix.

Les lignes de niveau (ou courbes isocontours) en arrière-plan indiquent les zones où la fonction de coût prend la même valeur. Les points rouges marquent les positions successives des paramètres (θ_0, θ_1) à chaque itération de l'algorithme. On observe clairement que ces points suivent un chemin progressif en spirale vers le minimum global de la fonction.

Cette trajectoire illustre comment l'algorithme ajuste ses paramètres pour minimiser l'erreur entre les prédictions et les valeurs réelles. Elle met en évidence la convergence du modèle et la manière dont la descente de gradient se déplace dans le paysage de la fonction de coût.

Ce type de visualisation est fondamental pour comprendre la dynamique d'apprentissage du modèle et les effets du taux d'apprentissage (learning rate) sur la vitesse de convergence.

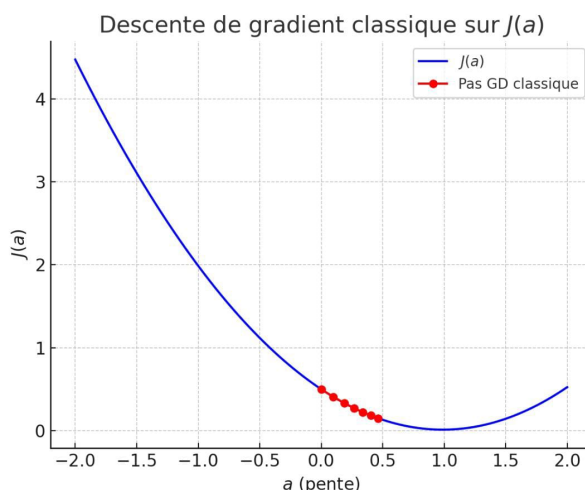


FIGURE 3.4 – Descente de gradient

Ce graphe représente la trajectoire de la descente de gradient classique appliquée à la fonction de coût, dans le cadre d'un modèle de régression linéaire entre la surface des maisons et leur prix.

La courbe bleue montre l'évolution de la fonction de coût en fonction de la pente. Les points rouges illustrent les mises à jour successives du paramètre à chaque itération. On observe une descente régulière vers le minimum de $J(a)$, ce qui reflète la stabilité et la convergence progressive de l'algorithme lorsque toutes les données sont prises en compte à chaque étape.

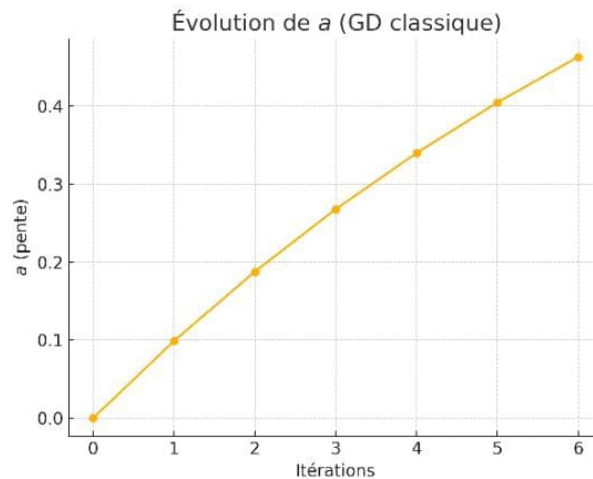


FIGURE 3.5 – Descente de gradient

Ce graphique montre l'évolution du paramètre au fil des itérations lors de l'application de la descente de gradient classique sur un modèle de régression linéaire.

On observe une progression régulière et monotone de la valeur de a , ce qui traduit la stabilité de l'algorithme. Chaque mise à jour est effectuée en prenant en compte l'ensemble des données, ce qui permet une convergence fluide vers la valeur optimale du paramètre.

3.3 Descente de gradient stochastique (SGD)

3.3.1 Propriétés des erreurs stochastiques

Dans un modèle de régression linéaire stochastique, le terme d'erreur ε représente l'incertitude ou la variation non expliquée par la variable explicative X . Pour garantir la validité et l'efficacité des estimations, les erreurs doivent respecter plusieurs propriétés fondamentales :

- **Espérance nulle** : $E(\varepsilon) = 0$. Cela garantit que la moyenne des différences entre les valeurs observées Y et les valeurs prédites $\hat{Y}$ est nulle :

$$E(\varepsilon) = E(Y - \hat{Y}) = E(Y - (\theta_0 + \theta_1 X)) = 0$$

- **Variance constante (homoscédasticité)** : $\text{Var}(\varepsilon|X) = \sigma^2$. Cela signifie que la variabilité des erreurs est uniforme quel que soit le niveau de X , et peut être formulé ainsi :

$$\text{Var}(\varepsilon|X) = E[(\varepsilon - E(\varepsilon))^2] = \sigma^2$$

- **Indépendance des erreurs** : $E(\varepsilon_i \varepsilon_j) = 0$ pour $i \neq j$. Cela signifie que les erreurs

associées à des observations différentes ne sont pas corrélées :

$$\text{Cov}(\varepsilon_i, \varepsilon_j) = 0 \quad \text{pour } i \neq j$$

- **Indépendance entre X et ε** : $\text{Cov}(X, \varepsilon) = 0$. Cela garantit que les erreurs ne sont pas liées aux valeurs des variables explicatives X :

$$E(\varepsilon|X) = 0$$

- **Distribution normale des erreurs** : $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Cela signifie que les erreurs suivent une distribution normale avec une moyenne de zéro et une variance constante σ^2 :

$$f(\varepsilon) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\varepsilon^2}{2\sigma^2}\right)$$

3.3.2 Estimation des paramètres

Dans un modèle de régression linéaire stochastique, l'objectif est d'estimer les paramètres β qui minimisent la différence entre les valeurs observées y et les valeurs prédites $X\beta$. La méthode des **moindres carrés ordinaires (MCO)** est utilisée pour cela.

Fonction coût à minimiser :

$$J(\beta) = (y - X\beta)^\top (y - X\beta)$$

Solution optimale (équation normale) :

$$\hat{\beta} = (X^\top X)^{-1} X^\top y$$

où $\hat{\beta}$ est l'estimateur des moindres carrés.

3.3.3 Propriétés des estimateurs

Sous les hypothèses classiques du modèle (normalité, homoscedasticité, indépendance), l'estimateur MCO $\hat{\beta}$ vérifie :

- **Non-biaisé** :

$$\mathbb{E}[\hat{\beta}] = \beta$$

- **Variance minimale** : $\hat{\beta}$ est le meilleur estimateur linéaire sans biais (BLUE) selon le théorème de Gauss-Markov.

- **Distribution normale** :

$$\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(X^\top X)^{-1})$$

Ces propriétés permettent d'effectuer des tests statistiques et de construire des intervalles de confiance.

- **Test de Student (t-test)** : Ce test permet de vérifier si un coefficient de régression β_j est significativement différent de zéro. L'hypothèse nulle H_0 est :

$$H_0 : \beta_j = 0$$

On calcule la statistique de test comme suit :

$$t = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)}$$

où $\text{SE}(\hat{\beta}_j)$ est l'erreur standard de l'estimateur $\hat{\beta}_j$. Si la valeur absolue de t est supérieure à une valeur critique donnée (basée sur la distribution t de Student), nous rejetons l'hypothèse nulle.

- **Intervalle de confiance** : Un intervalle de confiance à $100(1 - \alpha)\%$ pour un paramètre β_j peut être calculé comme suit :

$$\hat{\beta}_j \pm t_{\alpha/2, n-k} \cdot \text{SE}(\hat{\beta}_j)$$

où $t_{\alpha/2, n-k}$ est la valeur critique de la distribution t de Student avec $n - k$ degrés de liberté, n étant le nombre d'observations et k le nombre de paramètres estimés.

Ces outils sont essentiels pour évaluer la précision des estimations et la fiabilité des conclusions tirées du modèle.

3.3.4 Évaluation du modèle

L'évaluation d'un modèle de régression linéaire repose sur plusieurs indicateurs permettant d'apprécier la qualité de l'ajustement aux données ainsi que les performances prédictives du modèle. Voici les principaux critères utilisés :

- **Coefficient de détermination (R^2)** : Le coefficient R^2 mesure la proportion de la variance des données expliquée par le modèle. Il est défini par :

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

où y_i sont les valeurs observées, $\hat{y}_i$ les valeurs prédites, et $\bar{y}$ la moyenne des y_i . Une valeur de R^2 proche de 1 indique un ajustement satisfaisant du modèle.

- **Erreur quadratique moyenne (MSE)** : La MSE (Mean Squared Error) quantifie l'erreur moyenne entre les prédictions du modèle et les valeurs réelles :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Plus la MSE est faible, plus les prédictions sont précises.

- **Test de significativité globale (test F)** : Ce test permet d'évaluer si l'ensemble

des variables explicatives apporte une contribution significative au modèle. L'hypothèse nulle $H_0 : \beta_1 = \beta_2 = \dots = \beta_k = 0$ signifie que les prédicteurs n'ont aucun effet. La statistique F s'écrit :

$$F = \frac{(R^2/k)}{((1 - R^2)/(n - k - 1))}$$

Si F dépasse une valeur seuil, on rejette H_0 , indiquant que le modèle est globalement significatif.

- **Analyse des résidus** : Les résidus $e_i = y_i - \hat{y}_i$ permettent de vérifier plusieurs hypothèses fondamentales : linéarité, homoscédasticité (variance constante des erreurs), et normalité. Une analyse graphique (diagramme des résidus) est souvent utilisée pour détecter d'éventuelles anomalies ou patterns.

3.3.5 Application de SGD

Dans le cas de la régression linéaire simple avec la fonction de coût suivante :

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m (y_i - (ax_i + b))^2$$

Le gradient s'écrit comme :

$$\frac{\partial J}{\partial a} = -\frac{1}{m} \sum_{i=1}^m x_i \cdot (y_i - (ax_i + b)), \quad \frac{\partial J}{\partial b} = -\frac{1}{m} \sum_{i=1}^m (y_i - (ax_i + b))$$

En descente de gradient **stochastique**, au lieu de calculer la somme sur tous les exemples, on met à jour les paramètres à chaque itération en utilisant une seule observation :

$$a \leftarrow a - \alpha \cdot [-x_i \cdot (y_i - (ax_i + b))], \quad b \leftarrow b - \alpha \cdot [-(y_i - (ax_i + b))]$$

Exemple : soit les paramètres initiaux $a = 10$, $b = 100$, et $\alpha = 0,01$. On considère la première observation : $x_1 = 50$, $y_1 = 850$.

$$h(x_1) = ax_1 + b = 10 \times 50 + 100 = 600$$

$$\text{Erreur : } y_1 - h(x_1) = 850 - 600 = 250$$

$$\Delta a = \alpha \cdot x_1 \cdot (y_1 - h(x_1)) = 0,01 \times 50 \times 250 = 125$$

$$\Delta b = \alpha \cdot (y_1 - h(x_1)) = 0,01 \times 250 = 2,5$$

Mise à jour des paramètres :

$$a \leftarrow 10 + 125 = \boxed{135}, \quad b \leftarrow 100 + 2,5 = \boxed{102,5}$$

Ainsi, après une seule mise à jour basée sur la première observation, les nouveaux paramètres sont $a = 135$ et $b = 102,5$. La descente de gradient stochastique continue ensuite avec les autres observations, mises à jour une par une.

Illustration graphique suggérée :

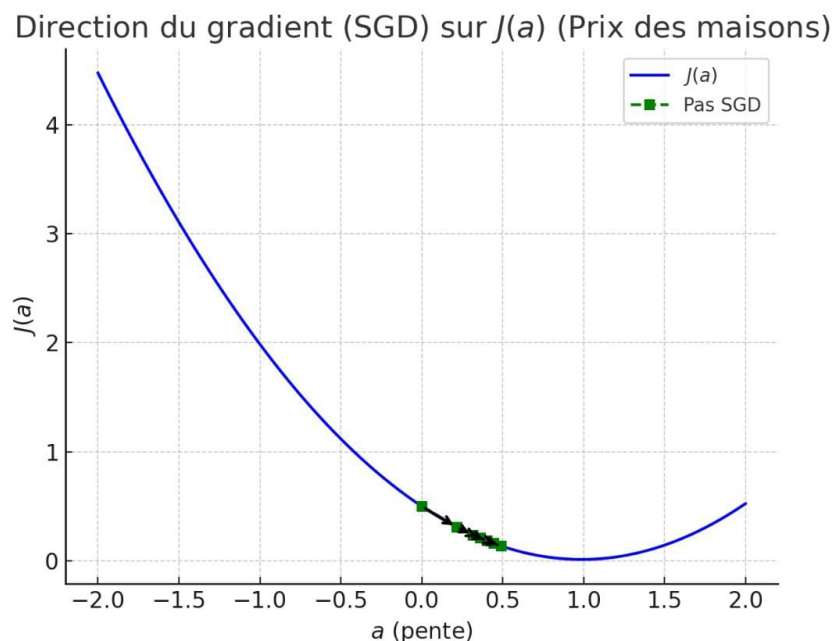


FIGURE 3.6 – Visualisation de la fonction de coût $J(\theta)$ et des étapes du SGD

Ce graphique représente la fonction de coût $J(\theta)$ en une dimension avec seulement un paramètre θ (dans notre exemple, le modèle est simplifié à une seule variable d'entrée normalisée).

La courbe bleue montre le profil de la fonction de coût, avec une forme convexe typique d'une régression linéaire simple.

Les points rouges illustrent les différentes étapes de mise à jour du paramètre θ par l'algorithme de descente de gradient stochastique (SGD). Contrairement à la version classique, le SGD effectue des mises à jour après avoir vu un seul exemple à la fois, ce qui introduit une certaine instabilité dans le parcours.

On remarque que les sauts ne suivent pas une ligne directe vers le minimum. Ils sont irréguliers et parfois bruités, ce qui est une caractéristique naturelle du comportement stochastique.

Cette visualisation permet de mieux comprendre les avantages et les limites du SGD : il peut converger plus rapidement dans certains cas, mais nécessite un bon ajustement du taux d'apprentissage pour éviter des oscillations trop importantes.

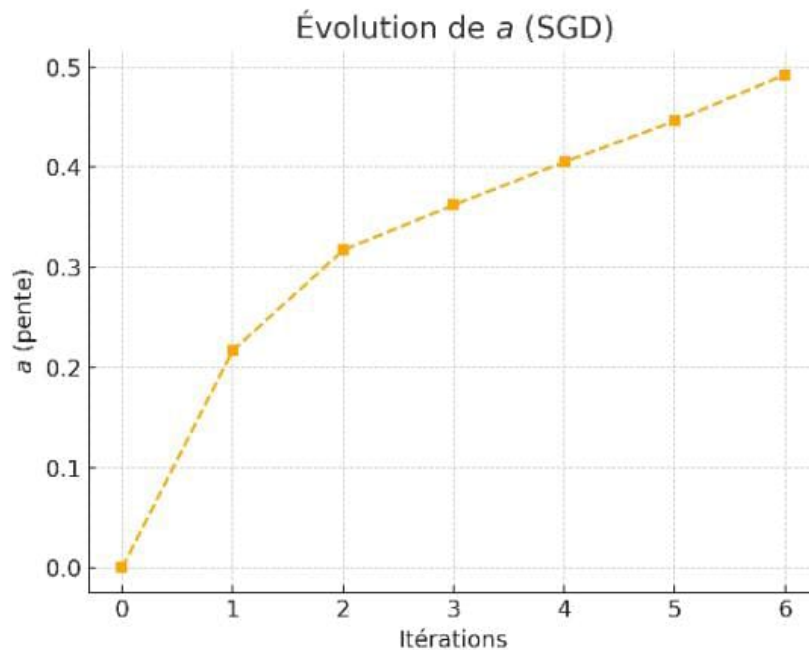


FIGURE 3.7 – Visualisation de la fonction de coût $J(\theta)$ et des étapes du SGD

Ce graphique montre l'évolution du paramètre θ au cours des itérations successives de l'algorithme de descente de gradient stochastique (SGD).

À chaque itération, le modèle met à jour la valeur de θ en fonction d'un seul exemple de données choisi aléatoirement. Contrairement à la descente de gradient classique, où la trajectoire est généralement lisse et directe, les valeurs de θ ici fluctuent d'un pas à l'autre de manière irrégulière.

On observe que les points ne suivent pas une progression parfaitement stable vers l'optimum.

Ces oscillations sont dues à la nature aléatoire du SGD, mais elles permettent parfois au modèle d'échapper aux minima locaux et de converger plus rapidement.

Ce type de visualisation est utile pour analyser la stabilité de l'apprentissage, ajuster le taux d'apprentissage et comparer les comportements entre différentes stratégies d'optimisation.

3.4 Comparaison entre la descente de gradient classique (GD) et la descente de gradient stochastique (SGD)

Les graphiques suivants permettent de visualiser la différence de comportement entre la descente de gradient classique et stochastique.

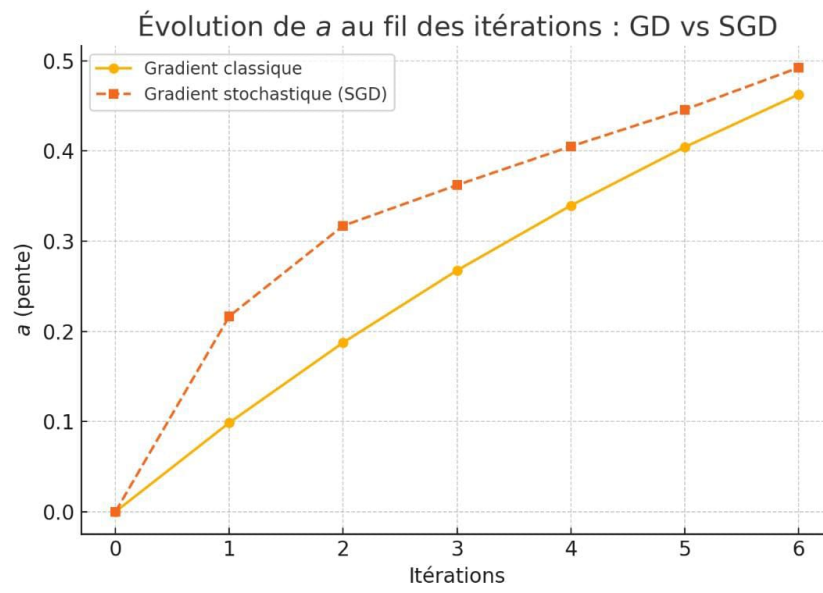


FIGURE 3.8 – Comparaison des trajectoires GD vs SGD sur la fonction de coût

Cette figure illustre les trajectoires empruntées par le GD classique et le SGD sur la fonction de coût .

La courbe bleue représente la fonction de coût. Les points rouges montrent les mises à jour de par le GD classique, et les points verts montrent celles du SGD.

Le GD classique suit une descente stable et directe vers le minimum, tandis que le SGD fluctue autour de la courbe avant de s'approcher du minimum.

Cette représentation met en évidence le compromis entre stabilité (GD) et rapidité initiale (SGD).

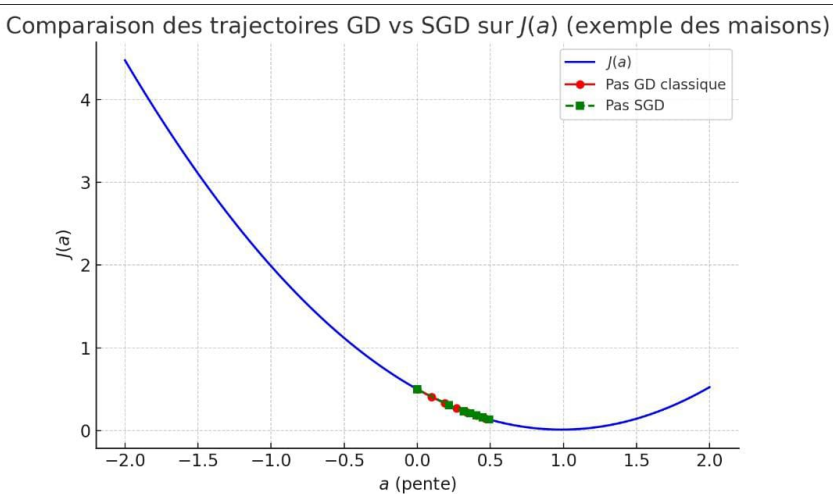


FIGURE 3.9 – Évolution du paramètre au fil des itérations : GD vs SGD

Ce graphique compare l'évolution du paramètre au fil des itérations pour deux méthodes : la descente de gradient classique (en jaune) et la descente de gradient stochastique (en orange).

On observe que le GD classique progresse de manière régulière, tandis que le SGD présente des oscillations dues à sa nature aléatoire. Cependant, le SGD converge plus rapidement au départ, bien que son chemin soit moins stable.

Conclusion

Critère	Descente de Gradient Classique (GD)	Descente de Gradient Stochastique (SGD)
Stabilité	Très stable avec une trajectoire régulière et contrôlée	Moins stable, souvent sujette à des fluctuations (bruit)
Rapidité par itération	Lente (car utilise toutes les données à chaque itération)	Très rapide (mise à jour basée sur un seul exemple)
Convergence globale	Convergence progressive et fiable vers le minimum global	Convergence plus rapide au début, mais parfois instable
Sensibilité au taux d'apprentissage α	Moins sensible aux variations de α	Très sensible, nécessite un bon réglage de α
Coût de calcul par itération	Élevé : calcul sur tout le dataset à chaque pas	Faible : une seule observation suffit pour mettre à jour

TABLE 3.1 – Comparaison détaillée entre la descente de gradient classique et la descente stochastique

Le tableau précédent présente une comparaison qualitative entre la descente de gradient classique (GD) et la descente de gradient stochastique (SGD) selon plusieurs critères importants :

-**Stabilité** : La méthode GD est très stable car elle utilise l'ensemble des données à chaque itération, ce qui permet une progression régulière vers le minimum. À l'inverse, le SGD présente plus de fluctuations dues à l'utilisation aléatoire d'exemples individuels.

-**Rapidité par itération** : Chaque itération du GD nécessite un calcul global sur toutes les données, ce qui ralentit son exécution. En revanche, le SGD effectue une mise à jour rapide basée sur un seul exemple, ce qui le rend bien plus léger.

-**Convergence globale** : Le GD converge de manière progressive et stable vers le minimum global, tandis que le SGD converge plus rapidement au début, mais de façon moins régulière.

-**Sensibilité au taux d'apprentissage** : Le GD est relativement tolérant à un mauvais choix de taux d'apprentissage, alors que le SGD est beaucoup plus sensible et nécessite un réglage précis pour éviter les divergences ou les lenteurs.

-**Coût de calcul** : Le coût d'une mise à jour dans le GD est élevé, car elle implique tous les exemples. Le SGD, de son côté, est bien plus efficace en termes de calcul car il n'utilise qu'un seul point de données à chaque mise à jour.

La différence principale réside dans la manière dont le gradient est calculé à chaque itération. Avec le GD classique, le gradient est évalué sur l'ensemble du jeu de données, ce qui garantit une trajectoire stable vers le minimum, mais au prix d'un coût de calcul important. À l'inverse, le SGD utilise un seul exemple à chaque mise à jour, rendant les itérations plus rapides, mais le chemin suivi est souvent bruité et moins régulier.

En résumé, le GD est préférable lorsque les données sont de taille modérée et que la stabilité est cruciale, tandis que le SGD est plus adapté aux grands ensembles de données ou lorsque la rapidité d'apprentissage est prioritaire.

Chapitre 4

Étude mathématique de Classification

4.1 Étude Mathématique du SVM Linéaire

4.1.1 Objectif de l'algorithme SVM linéaire

L'objectif du *Support Vector Machine (SVM)* est de trouver un **hyperplan optimal** qui sépare linéairement deux classes de données tout en maximisant la **marge** entre elles.

On considère un ensemble d'entraînement :

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^l, \quad x_i \in \mathbb{R}^n, \quad y_i \in \{-1, +1\}$$

4.1.2 Hyperplan de séparation

Un hyperplan est défini par :

$$f(x) = \langle w, x \rangle + b = 0$$

où :

- $w \in \mathbb{R}^n$ est le vecteur normal à l'hyperplan,
- $b \in \mathbb{R}$ est le biais.

L'objectif est de satisfaire :

$$y_i(\langle w, x_i \rangle + b) \geq 1 \quad \text{pour tout } i$$

4.1.3 Maximisation de la marge

La marge est la distance entre les deux hyperplans :

$$H_1 : \langle w, x \rangle + b = 1 \quad \text{et} \quad H_2 : \langle w, x \rangle + b = -1$$

La distance entre ces deux hyperplans est :

$$\text{Marge} = \frac{2}{\|w\|}$$

Maximiser la marge revient donc à minimiser $\|w\|^2$.

4.1.4 Problème d'optimisation primal

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

sous la contrainte : $y_i(\langle w, x_i \rangle + b) \geq 1, \quad \forall i$

4.1.5 Formulation duale (multiplicateurs de Lagrange)

On introduit des multiplicateurs $\alpha_i \geq 0$, et on obtient le problème dual :

$$\max_{\alpha} \sum_{i=1}^l \alpha_i - \frac{1}{2} \sum_{i,j=1}^l \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$$

sous la contrainte : $\sum_{i=1}^l \alpha_i y_i = 0, \quad \alpha_i \geq 0$

Une fois les α_i optimaux obtenus, on a :

$$w = \sum_{i=1}^l \alpha_i y_i x_i \quad \text{et} \quad b = y_k - \langle w, x_k \rangle$$

pour un vecteur de support x_k .

4.1.6 Fonction de décision

La fonction de décision finale est :

$$f(x) = \sum_{i=1}^l \alpha_i y_i \langle x_i, x \rangle + b$$

Les points pour lesquels $\alpha_i > 0$ sont appelés **vecteurs de support**.

4.1.7 Exemple numérique simple

1. Ensemble de données

Considérons un ensemble de données linéairement séparables dans $\mathbb{R}^2$:

i	$x_i = (x_{i1}, x_{i2})$	y_i
1	(2, 2)	+1
2	(4, 4)	+1
3	(2, 0)	-1
4	(0, 0)	-1

2. Objectif de l'apprentissage SVM

L'objectif est de déterminer un hyperplan de séparation de la forme :

$$f(x) = w \cdot x + b = 0$$

tel que :

$$y_i(w \cdot x_i + b) \geq 1, \quad \forall i$$

et de minimiser :

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

ce qui permet de maximiser la marge entre les deux classes.

3. Choix d'un hyperplan candidat

On propose :

$$w = (1, 1), \quad b = -3 \Rightarrow f(x) = x_1 + x_2 - 3$$

4. Vérification des contraintes

On vérifie que chaque point satisfait :

$$y_i(w \cdot x_i + b) \geq 1$$

— Pour $x_1 = (2, 2), y_1 = +1$:

$$f(x_1) = 2 + 2 - 3 = 1 \Rightarrow y_1 f(x_1) = 1 \geq 1$$

— Pour $x_2 = (4, 4), y_2 = +1$:

$$f(x_2) = 8 - 3 = 5 \Rightarrow y_2 f(x_2) = 5 \geq 1$$

— Pour $x_3 = (2, 0), y_3 = -1$:

$$f(x_3) = 2 - 3 = -1 \Rightarrow y_3 f(x_3) = (+1) \geq 1$$

— Pour $x_4 = (0, 0), y_4 = -1$:

$$f(x_4) = -3 \Rightarrow y_4 f(x_4) = (+3) \geq 1$$

Toutes les contraintes sont vérifiées.

5. Calcul de la marge

La norme de w est :

$$\|w\| = \sqrt{1^2 + 1^2} = \sqrt{2} \Rightarrow \text{Marge} = \frac{2}{\|w\|} = \frac{2}{\sqrt{2}} = \sqrt{2}$$

6. Identification des vecteurs de support

Un vecteur de support est un point x_i tel que :

$$y_i(w \cdot x_i + b) = 1$$

- $x_1 = (2, 2) \Rightarrow f(x_1) = 1 \Rightarrow$ vecteur de support
- $x_3 = (2, 0) \Rightarrow f(x_3) = -1, y_3 = -1 \Rightarrow y_3 f(x_3) = 1 \Rightarrow$ vecteur de support

Donc, les vecteurs de support sont :

$$x_1 = (2, 2), \quad x_3 = (2, 0)$$

7. Résumé final

Élément	Valeur
Hyperplan séparateur	$f(x) = x_1 + x_2 - 3$
Norme du vecteur w	$\ w\ = \sqrt{2}$
Marge entre les classes	$\frac{2}{\ w\ } = \sqrt{2}$
Vecteurs de support	$(2, 2)$ et $(2, 0)$
Fonction de décision	$f(x) = \text{sign}(x_1 + x_2 - 3)$

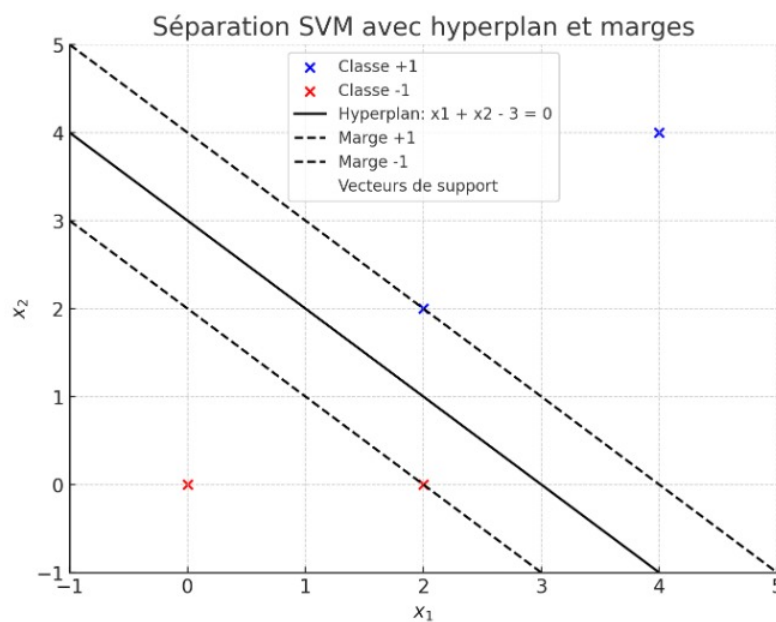


FIGURE 4.1 – Courbe de separation svm

Interprétation de la figure :

La ligne noire continue représente l'**hyperplan de séparation** défini par l'équation :

$$x_1 + x_2 - 3 = 0$$

Les deux lignes noires pointillées correspondent aux **marges** situées à une distance de :

$$\frac{1}{\|w\|} = \frac{1}{\sqrt{2}}$$

de part et d'autre de l'hyperplan.

Les points entourés en vert sont les **vecteurs de support**, à savoir :

$$x_1 = (2, 2) \quad \text{et} \quad x_3 = (2, 0)$$

Les points bleus correspondent aux exemples de la classe $+1$, tandis que les points rouges correspondent à ceux de la classe -1 .

Chapitre 5

Technologies et Environnements de Travail

5.1 Outils Utilisés

5.1.1 Python

Python est un langage de programmation efficace et largement utilisé qui est devenu dernièrement le langage de prédilection des data scientists. C'est un langage convivial pour les débutants en raison de sa syntaxe simplifiée, proche du langage naturel. Il a été conçu pour écrire un code clair et logique pour les projets de petite et grande envergure. Il est largement utilisé dans l'industrie pour le développement de logiciels, la science des données, l'intelligence artificielle et l'apprentissage automatique. Sa vaste communauté active contribue régulièrement à l'enrichissement de ses bibliothèques, ce qui garantit un support constant et une évolution rapide. Python est également apprécié pour sa portabilité, étant compatible avec de nombreux systèmes d'exploitation.

Le choix de Python dans notre projet s'explique par la richesse de son écosystème, notamment les bibliothèques dédiées au Machine Learning supervisé telles que **Scikit-Learn**, **Pandas** ou **NumPy**. De plus, son intégration fluide avec la distribution Anaconda nous a permis d'installer et de gérer facilement ces outils. Dans notre travail, Python a été utilisé pour le traitement et la préparation des données, la construction des modèles d'apprentissage supervisé, ainsi que l'évaluation des performances.



FIGURE 5.1 – Logo de Python

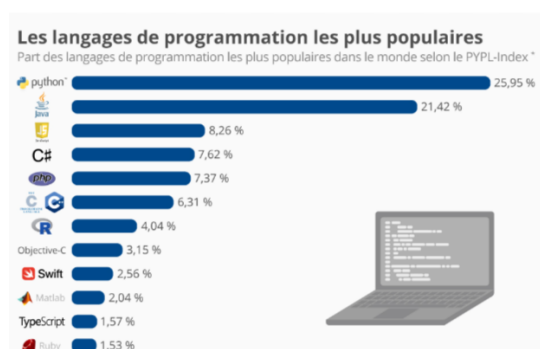


FIGURE 5.2 – Interface d'Anaconda Navigator

Python occupe aujourd'hui la première place parmi les langages de programmation les plus utilisés dans le monde, devant des langages historiques comme Java, JavaScript et C#. Cette popularité s'explique par sa simplicité, sa polyvalence et sa forte intégration dans les domaines émergents tels que la science des données, l'intelligence artificielle et le développement web. D'autres langages comme C++, PHP ou encore Swift conservent une présence importante, chacun étant adapté à des contextes spécifiques de développement. On constate également une montée progressive de langages modernes comme TypeScript et Go, portés par les besoins croissants en performance et en développement d'applications web complexes. Cette répartition reflète les tendances actuelles du secteur technologique et souligne l'importance pour les développeurs de maîtriser des outils adaptés à leurs domaines d'application.

5.1.2 Anaconda

Anaconda est un outil dont la distribution est libre et open source, conçu pour le développement en Python et en R. Il est largement utilisé dans les domaines de la science des données, de l'intelligence artificielle et du Machine Learning. Cette distribution scientifique de Python intègre de nombreux packages essentiels à l'analyse et à la manipulation des données, tout en offrant un système de gestion d'environnements virtuels performant.



FIGURE 5.3 – Jupiter

Cet outil est Utilisé par plus de 20 millions de personnes dans le monde, Anaconda comprend notamment :

- une installation complète de l'environnement Python ;
- des environnements de développement intégrés (IDE) modernes tels que Jupyter ou Spyder ;
- des bibliothèques de science des données telles que Pandas, NumPy, Scikit-Learn ;
- l'outil **Conda** pour la gestion des environnements virtuels et des dépendances.

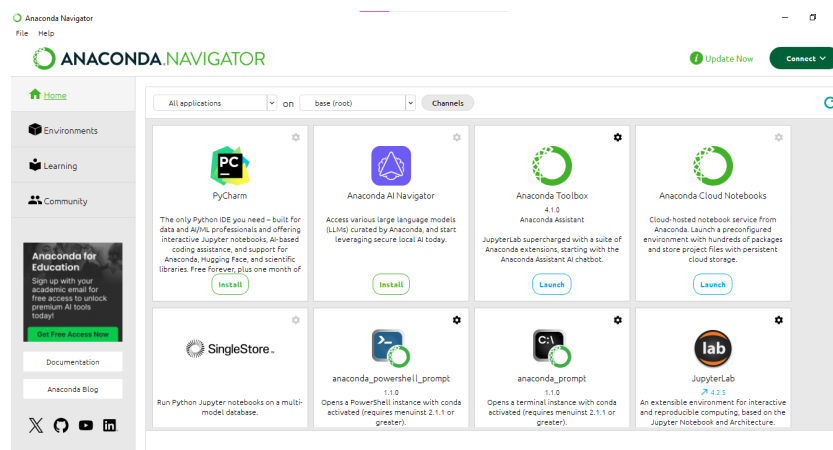


FIGURE 5.4 – Interface d'Anaconda Navigator

Grâce à l'ensemble de ces outils et à l'écosystème Python, Anaconda permet une collecte, une transformation et une analyse efficace des données à grande échelle. Par ailleurs, Jupyter, l'un des environnements de développement inclus, prend désormais en charge plus de 40 langages de programmation.

5.1.3 Jupyter

Jupyter Notebook est une application web interactive qui permet de créer et de partager des documents mêlant code exécutable, équations, visualisations et commentaires textuels. Elle s'appuie sur le langage Python, mais prend également en charge de nombreux autres langages grâce au système de noyaux (kernels), tels que R ou Julia. Cet environnement est particulièrement apprécié dans le domaine de la science des données, du calcul scientifique et de l'intelligence artificielle, car il favorise une approche itérative et visuelle du développement.

Grâce à sa structure en cellules, Jupyter facilite l'expérimentation : chaque portion de code peut être exécutée indépendamment, avec un affichage immédiat des résultats, ce qui permet une exploration rapide et intuitive des données. Il offre également la possibilité d'intégrer des graphiques interactifs, des tableaux, du Markdown, des équations en $\text{\LaTeX}$, ce qui le rend idéal pour la documentation et la communication des analyses.

Dans notre projet, Jupyter a été utilisé comme environnement principal de développement. Il nous a permis de structurer notre travail de manière claire, d'analyser les données progressivement, de tester et d'ajuster les modèles de Machine Learning supervisé, tout en gardant une trace complète et compréhensible du processus, facilitant ainsi la reproductibilité et la présentation finale des résultats.



FIGURE 5.5 – Jupiter

5.2 Bibliothèques Utilises

5.2.1 Matplotlib et NumPy

Matplotlib est une bibliothèque Python open-source créée en 2002 par le neurobiologiste John Hunter. À l'origine, son objectif était de visualiser les signaux cérébraux de patients épileptiques, en reproduisant les capacités graphiques de MATLAB, mais en utilisant Python. Depuis le décès de John Hunter en 2012, la bibliothèque a continué d'évoluer grâce à de nombreux contributeurs de la communauté open-source.

Matplotlib permet de créer facilement des visualisations de données de haute qualité : courbes, histogrammes, diagrammes en barres, nuages de points, et bien d'autres types de graphiques, le tout en quelques lignes de code. C'est un outil complet qui rend possible la génération de graphiques détaillés et personnalisables. Particulièrement utile pour ceux qui travaillent avec Python ou NumPy, Matplotlib s'utilise aussi bien dans les scripts Python que dans les notebooks Jupyter, les serveurs web ou les interfaces graphiques. Grâce à ses API flexibles, elle peut être intégrée dans des applications pour produire des visualisations dynamiques et interactives.



FIGURE 5.6 – Exemple de visualisation avec Matplotlib

Qu'est-ce que PyPlot ?

Pyplot est un module de Matplotlib qui propose plusieurs fonctions simples permettant d'ajouter des éléments tels que des lignes, des images ou du texte aux axes d'un graphique. Son interface est très pratique, ce qui explique pourquoi ce module est largement utilisé.

NumPy est une bibliothèque essentielle pour le calcul scientifique en Python. Elle offre un type de données puissant, le tableau multidimensionnel (`ndarray`), permettant de traiter efficacement des ensembles numériques homogènes. Grâce à une vaste collection de fonctions mathématiques, logiques et statistiques, NumPy facilite la manipulation de données complexes, les opérations sur les matrices, ainsi que les transformations et simulations diverses.

Très utilisée en science des données, elle remplace souvent des outils comme MATLAB, tout en servant de fondation à d'autres bibliothèques comme TensorFlow ou Scikit-learn. NumPy est également appréciée pour ses performances nettement supérieures aux listes Python classiques et pour sa syntaxe concise, souvent utilisée via l'alias `np`. Elle constitue ainsi un outil incontournable pour tous ceux qui travaillent en analyse de données, en intelligence artificielle ou en modélisation scientifique avec Python.

Son efficacité repose sur une structure optimisée en langage C, ce qui permet une exécution rapide des opérations. NumPy permet aussi une meilleure gestion de la mémoire, ce qui est essentiel dans les projets manipulant de grandes quantités de données. En résumé, c'est un outil puissant, flexible et indispensable dans l'écosystème Python scientifique.



FIGURE 5.7 – Illustration du fonctionnement de NumPy

5.2.2 Pandas

Pandas est une bibliothèque puissante et incontournable pour l'analyse et la manipulation de données en Python. Elle repose sur deux structures principales : les Series (tableaux unidimensionnels) et les DataFrames (tableaux bidimensionnels similaires aux feuilles Excel ou aux tables SQL), qui permettent d'organiser, nettoyer, transformer et analyser les données de manière simple et efficace.

Grâce à Pandas, il est possible de lire des fichiers CSV, Excel, JSON, ou encore des bases de données SQL en quelques lignes de code. Elle propose de nombreuses fonctionnalités pour filtrer, trier, grouper ou encore fusionner des ensembles de données avec une grande souplesse.

Très utilisée en data science, en finance, en statistiques ou en ingénierie, Pandas permet de gagner un temps précieux dans toutes les phases de traitement de données. Son intégration fluide avec NumPy, Matplotlib ou Scikit-learn en fait un pilier de l'écosystème Python. Pour l'utiliser, il suffit généralement de l'importer sous l'alias `pd`, ce qui facilite son utilisation au quotidien.



FIGURE 5.8 – Pandas

- **Lecture et écriture de fichiers** : Chargement et sauvegarde des données au format CSV, Excel, JSON, SQL, etc.
- **Filtrage et sélection des données** : Accès aux lignes et colonnes via `.loc`, `.iloc`, ou des conditions logiques.
- **Nettoyage des données** : Gestion des valeurs manquantes avec `.isnull()`, `.fillna()`, `.dropna()`.
- **Transformation des données** : Utilisation de `.apply()`, `.map()`, `.replace()` pour appliquer des fonctions personnalisées.
- **Tri et réorganisation** : Tri des données avec `.sort_values()`, *reindexation avec*

5.2.3 Scikit-Learn

Scikit-Learn est une bibliothèque Python dédiée au machine learning. Son interface simple et unifiée permet d'accéder à de nombreux algorithmes de classification, de régression, de clustering, de réduction de dimensionnalité et de sélection de modèles. Scikit-

Learn est basée sur les bibliothèques NumPy, SciPy et matplotlib, qui sont des outils indispensables pour la manipulation de données et la visualisation en Python.

Scikit-Learn est une bibliothèque open source sous licence BSD (Berkeley Software Distribution License) réputée pour sa documentation complète et sa communauté active qui fournit un soutien et des ressources supplémentaires. Scikit-Learn permet de construire des modèles prédictifs en s'appuyant sur des algorithmes de régressions linéaire et logistique, des Support Vector Machine (SVM), des arbres de décision, des forêts aléatoires. Les modèles prédictifs sont utiles dans les domaines du diagnostic médical, la détection de fraudes, ou encore l'autonomie des véhicules par exemple.



FIGURE 5.9 – Scikit learn

Scikit-Learn est l'une des bibliothèques les plus populaires pour l'apprentissage automatique en Python. Voici quelques-uns de ses principaux avantages :

- **Simplicité d'utilisation** : Une API claire et cohérente qui permet d'entraîner, tester et évaluer des modèles en quelques lignes de code.
- **Large éventail d'algorithmes** : Intègre de nombreux algorithmes d'apprentissage supervisé et non supervisé tels que la régression, les SVM, les forêts aléatoires, le clustering, etc.
- **Excellente documentation** : Une documentation riche, bien structurée, avec des exemples concrets et des tutoriels pour tous les niveaux.
- **Intégration facile avec d'autres bibliothèques** : Compatible avec NumPy, Pandas, Matplotlib, et d'autres outils de l'écosystème scientifique Python.

Chapitre 6

Application

6.1 Régression

6.1.1 Application régression linéaire simple

Principe fondamental de la régression linéaire

La régression linéaire simple cherche à modéliser la relation entre une variable explicative x (par exemple : surface habitable **Gr Liv Area**) et une variable cible y (par exemple : prix de vente **SalePrice**) à l'aide d'une équation de droite :

$$y = b_0 + b_1x$$

où :

- b_0 est l'ordonnée à l'origine (biais),
- b_1 est le coefficient directeur (pente).

• Méthode des moindres carrés

On cherche les coefficients b_0 et b_1 qui minimisent la somme des carrés des écarts entre les valeurs observées y_i et les valeurs prédites $\hat{y}_i = b_0 + b_1x_i$.

Les formules utilisées sont :

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad b_0 = \bar{y} - b_1\bar{x}$$

Code Python pour le calcul des coefficients :

```
1 x_mean = np.mean(x)
2 y_mean = np.mean(y)
3
4 numerateur = np.sum((x - x_mean) * (y - y_mean))
5 denominateur = np.sum((x - x_mean) ** 2)
6 b1 = numerateur / denominateur
7 b0 = y_mean - b1 * x_mean
```

• Fonction de coût et minimisation

La fonction de coût (ou erreur quadratique moyenne) associée est :

$$J(b_0, b_1) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Elle est minimisée lorsque les coefficients b_0 et b_1 sont calculés à l'aide de la méthode des moindres carrés.

Visualisation de la droite ajustée (code Python) :

```
1 y_pred = b0 + b1 * x
2
3 plt.figure(figsize=(10, 6))
4 plt.scatter(x, y, color='blue', label='Données réelles', alpha=0.5)
5 plt.plot(x, y_pred, color='red', label='Régression linéaire')
6 plt.xlabel("Surface habitable (Gr Liv Area)")
7 plt.ylabel("Prix de vente (SalePrice)")
8 plt.title("Régression linéaire manuelle - Ames Housing Dataset")
9 plt.legend()
10 plt.grid(True)
11 plt.show()
```

Affichage de l'équation obtenue :

```
1 print(f"équation de la droite : y = {b0:.2f} + {b1:.2f} * x")
```

• Visualisation Graphique des Résultats

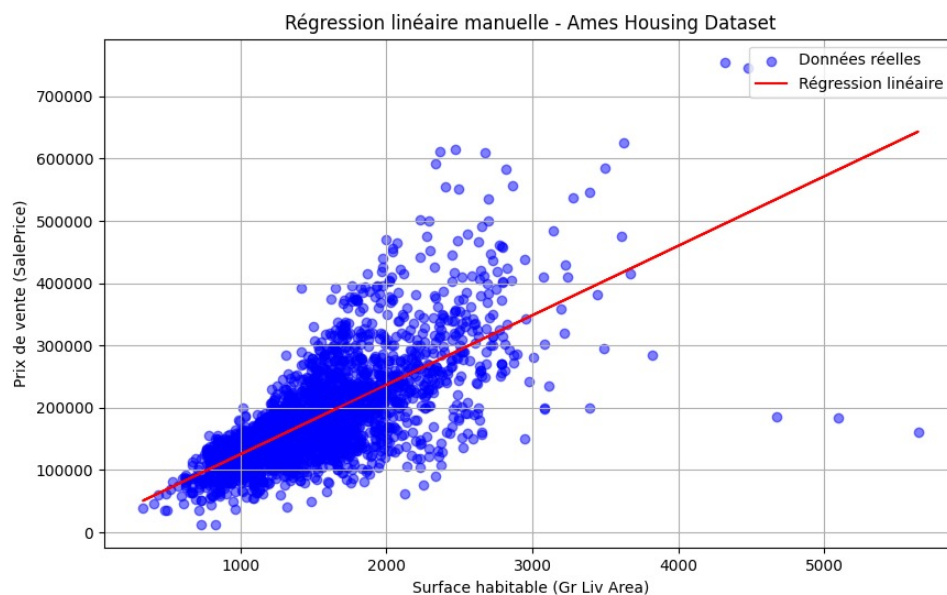


FIGURE 6.1 – Régression linéaire simple)

6.1.2 Application de la descente de gradient

• Chargement et normalisation des données

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Charger les donn\ees
6 data = pd.read_csv("AmesHousing.csv")
7 data = data[['Gr Liv Area', 'SalePrice']].dropna()
8
9 # S\eparer les variables explicative et cible
10 x = data['Gr Liv Area'].values
11 y = data['SalePrice'].values
12
13 # Normalisation de x
14 x = (x - np.mean(x)) / np.std(x)

```

Relation mathématique :

$$x_i^{\text{norm}} = \frac{x_i - \bar{x}}{\sigma_x}$$

• Initialisation des paramètres et hyperparamètres

```

1 b0 = 0 # intercept
2 b1 = 0 # slope
3
4 learning_rate = 0.01
5 epochs = 1000
6 n = len(x)
7 cost_history = []

```

• Descente de gradient classique (batch)

```

1 for epoch in range(epochs):
2     y_pred = b0 + b1 * x
3     error = y_pred - y
4     cost = (1/n) * np.sum(error ** 2)
5     cost_history.append(cost)

```

Relation mathématique :

$$J(b_0, b_1) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2, \quad \text{où } \hat{y}_i = b_0 + b_1 x_i$$

```

1 grad_b0 = (2/n) * np.sum(error)
2 grad_b1 = (2/n) * np.sum(error * x)

```

Relations mathématiques :

$$\frac{\partial J}{\partial b_0} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i), \quad \frac{\partial J}{\partial b_1} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$


```

1 b0 -= learning_rate * grad_b0
2 b1 -= learning_rate * grad_b1

```

Mise à jour des coefficients :

$$b_0 \leftarrow b_0 - \alpha \frac{\partial J}{\partial b_0}, \quad b_1 \leftarrow b_1 - \alpha \frac{\partial J}{\partial b_1}$$

• Prédiction finale

```

1 y_pred_final = b0 + b1 * x

```

• Visualisation des résultats

```

1 plt.figure(figsize=(10, 6))
2 plt.scatter(x, y, color='blue', label='Donn\ees r\eeelles', alpha=0.5)
3 plt.plot(x, y_pred_final, color='red', label='R\egression (descente de gradient)')
4 plt.xlabel("Surface habitable normalis\ee (Gr Liv Area)")
5 plt.ylabel("Prix de vente (SalePrice)")
6 plt.title("R\egression lin\eaire avec descente de gradient (Batch)")
7 plt.legend()
8 plt.grid(True)
9 plt.show()

```

• Visualisation de la fonction de coût

```

1 plt.plot(range(epochs), cost_history, color='green')
2 plt.xlabel("\Epochs")
3 plt.ylabel("Erreur quadratique moyenne (MSE)")
4 plt.title("\Evolution de la fonction de co\ut")
5 plt.grid(True)
6 plt.show()

```

• Affichage de l'équation estimée

```

1 print(f"\Equation estim\ee : y = {b0:.2f} + {b1:.2f} * x_normalis\ee")

```

• Visualisation Graphique des Résultats

Nous illustrons ci-dessous les résultats obtenus à l'aide de la descente de gradient :

- ▷ la droite de régression obtenue à partir des données normalisées,
- ▷ la courbe de convergence illustrant la diminution de l'erreur quadratique moyenne (MSE).

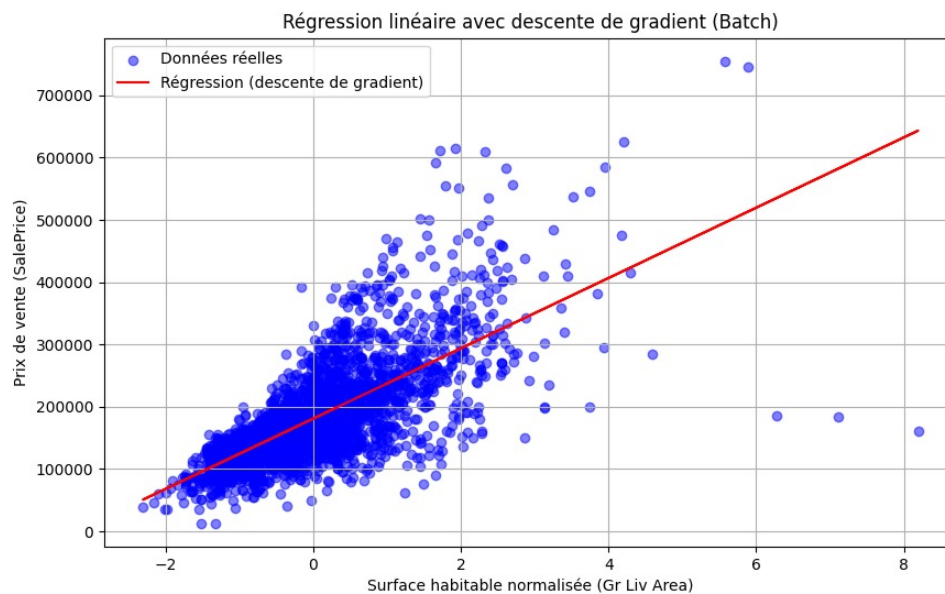


FIGURE 6.2 – Régression linéaire obtenue par descente de gradient

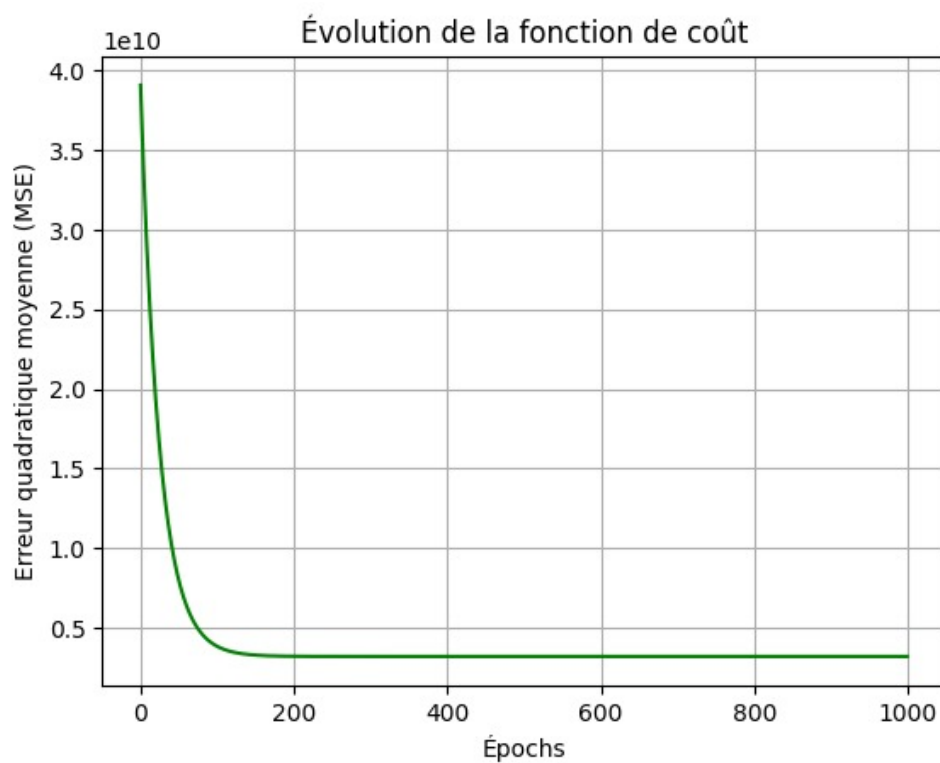


FIGURE 6.3 – Évolution de l'erreur quadratique moyenne (MSE) au fil des époques

6.1.3 Descente de Gradient Stochastique (SGD)

Pour ajuster les paramètres b_0 et b_1 du modèle $y = b_0 + b_1x$, on minimise la fonction de coût $J(b_0, b_1)$, généralement l'erreur quadratique moyenne. La descente de gradient stochastique consiste à mettre à jour les paramètres pour chaque exemple (x_i, y_i) selon les gradients locaux :

$$\begin{cases} b_0 \leftarrow b_0 - \alpha \cdot (b_0 + b_1x_i - y_i) \\ b_1 \leftarrow b_1 - \alpha \cdot (b_0 + b_1x_i - y_i) \cdot x_i \end{cases}$$

où α est le taux d'apprentissage. Contrairement à la descente de gradient classique (batch), la mise à jour se fait à chaque observation, ce qui accélère parfois la convergence et introduit de la variabilité bénéfique pour échapper aux minima locaux.

• Importation des Librairies

Nous importons les bibliothèques nécessaires pour la manipulation des données, les opérations numériques et la visualisation graphique.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
```

Listing 6.1 – Importation des librairies

- ◇ **pandas** : Facilite l'importation et la manipulation de structures de données tabulaires (DataFrames).
- ◇ **numpy** : Fournit des outils pour le calcul numérique efficace, notamment les opérations vectorielles et matricielles fondamentales à l'algèbre linéaire.
- ◇ **matplotlib.pyplot** : Permet la visualisation des données et des résultats du modèle, essentielle pour l'analyse graphique.

• Chargement et Préparation des Données

Nous chargeons un jeu de données et effectuons un prétraitement pour isoler les variables d'intérêt et les normaliser.

```
1 # Charger le dataset
2 data = pd.read_csv("AmesHousing.csv")
3
4 # Garde uniquement les colonnes utiles et supprime les lignes manquantes
5 data = data[['Gr Liv Area', 'SalePrice']].dropna()
6
7 # Extraire les variables
8 x = data['Gr Liv Area'].values
9 y = data['SalePrice'].values
```

```

10
11 # Normaliser x pour accélérer la convergence
12 x = (x - np.mean(x)) / np.std(x)

```

Listing 6.2 – Chargement et préparation des données

Explication Mathématique :

- ◇ On cherche à approximer $y_i = b_0 + b_1 x_i$ avec $x_i = \text{Gr Liv Area}$ et $y_i = \text{SalePrice}$.
- ◇ Le modèle considère n paires (x_i, y_i) issues du dataset après suppression des valeurs manquantes.
- ◇ La normalisation de x est effectuée par :

$$x_i^{\text{norm}} = \frac{x_i - \mu_x}{\sigma_x}$$

avec $\mu_x = \frac{1}{n} \sum_{i=1}^n x_i$ et $\sigma_x = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu_x)^2}$.

- ◇ Cette standardisation assure une meilleure stabilité numérique et une convergence plus rapide lors de l'optimisation par descente de gradient.

• Initialisation des Paramètres et Hyperparamètres

Nous initialisons les coefficients du modèle (b_0 , b_1) et définissons les paramètres de l'algorithme de descente de gradient.

```

1 # Initialiser les paramètres
2 b0 = 0 # intercept
3 b1 = 0 # slope
4
5 # Hyperparamètres
6 learning_rate = 0.01
7 epochs = 100
8 n = len(x)
9
10 # Historique de la fonction de coût (pour voir l'évolution)
11 cost_history = []

```

Listing 6.3 – Initialisation des paramètres et hyperparamètres

Explication Mathématique :

- ◇ On initialise b_0 et b_1 à 0 : $y_i \approx b_0 + b_1 x_i$.
- ◇ $\alpha = 0,01$ est le taux d'apprentissage utilisé dans la règle de mise à jour :

$$b_j \leftarrow b_j - \alpha \cdot \frac{\partial J}{\partial b_j}$$

- ◇ $n = |x|$ est la taille de l'échantillon d'apprentissage.
- ◇ $epochs = 100$ représente le nombre d'itérations globales sur le dataset.
- ◇ `cost_history` mémorise les valeurs successives de la fonction de coût $J(b_0, b_1)$.

• Entraînement du Modèle avec Descente de Gradient Stochastique (SGD)

L'algorithme SGD ajuste les paramètres b_0 et b_1 en minimisant la fonction de coût, qui est l'erreur quadratique moyenne. Contrairement à la descente de gradient par lot, la SGD met à jour les paramètres après chaque exemple d'entraînement.

```

1 # Boucle d'entraînement avec descente de gradient stochastique
2 for epoch in range(epochs):
3     total_cost = 0
4     for i in range(n):
5         xi = x[i]
6         yi = y[i]
7         y_pred = b0 + b1 * xi
8         error = y_pred - yi
9         # Mettre à jour les poids
10        b0 = b0 - learning_rate * error
11        b1 = b1 - learning_rate * error * xi
12        total_cost += error ** 2
13    cost = total_cost / n
14    cost_history.append(cost)

```

Listing 6.4 – Boucle d'entraînement SGD

Explication Mathématique :

◇ La fonction de coût est le MSE :

$$J(b_0, b_1) = \frac{1}{n} \sum_{i=1}^n (y_i - (b_0 + b_1 x_i))^2$$

◇ Pour chaque observation (x_i, y_i) , la prédiction est : $y_i^{\text{pred}} = b_0 + b_1 x_i$.

◇ L'erreur est donnée par $e_i = y_i^{\text{pred}} - y_i$.

◇ Les paramètres sont mis à jour selon la règle de descente de gradient stochastique :

$$b_0 \leftarrow b_0 - \alpha \cdot e_i$$

$$b_1 \leftarrow b_1 - \alpha \cdot e_i \cdot x_i$$

◇ À chaque époque, on cumule les e_i^2 pour calculer le coût moyen : $J = \frac{1}{n} \sum e_i^2$.

• Application

★ Visualisation des Résultats et Convergence

Après l'entraînement, nous visualisons la ligne de régression apprise par rapport aux données réelles et suivons l'évolution de la fonction de coût pour évaluer la convergence de l'algorithme.

```

1 # Pr dictions finales
2 y_pred_final = b0 + b1 * x
3
4 # Visualisation
5 plt.figure(figsize=(10, 6))
6 plt.scatter(x, y, color='blue', label='Donn es r elles', alpha=0.5)
7 plt.plot(x, y_pred_final, color='red', label='R gression SGD')
8 plt.xlabel("Surface habitable normalis e (Gr Liv Area)")
9 plt.ylabel("Prix de vente (SalePrice)")
10 plt.title("R gression lin aire avec descente de gradient stochastique")
11 plt.legend()
12 plt.grid(True)
13 plt.show()
14
15 # Tracer l'volution de la fonction de co t
16 plt.plot(range(epochs), cost_history, color='green')
17 plt.xlabel("pochs ")
18 plt.ylabel("Erreur quadratique moyenne (MSE)")
19 plt.title("Convergence de la descente de gradient stochastique")
20 plt.grid(True)
21 plt.show()

```

Listing 6.5 – Visualisation des résultats

Explication Mathématique :

- ◇ Le premier graphique (`plt.scatter` et `plt.plot`) représente la relation linéaire apprise : la droite $y_{pred} = b_0 + b_1 x_{norm}$ est superposée aux points de données observés (x_{norm}, y) .
- ◇ Le deuxième graphique (`plt.plot`) illustre la convergence de l'algorithme. Il montre comment $J(b_0, b_1)$ diminue à chaque époque, indiquant que l'algorithme se rapproche d'un minimum local de la fonction de coût. Une courbe qui s'aplatit suggère une convergence.

• Affichage de l'Équation Finale

Nous affichons l'équation de la droite de régression linéaire avec les coefficients b_0 et b_1 finaux.

```

1 # Affichage de l' quation
2 print(f" quation finale estim e : y = {b0:.2f} + {b1:.2f} * x_normalis ")

```

Listing 6.6 – Affichage de l'équation finale

Explication Mathématique :

- ◇ Les valeurs de b_0 et b_1 affichées sont les coefficients optimisés par la descente de gradient stochastique. L'équation représente le modèle linéaire appris : $Y = b_0 + b_1 \cdot X_{normalis}$.
- ◇ Cette équation peut être utilisée pour prédire le prix de vente (Y) pour une surface habitable normalisée ($X_{normalis}$) donnée.

• Visualisation Graphique des Résultats

Nous illustrons ci-dessous les résultats obtenus à l'aide de la descente de gradient stochastique :

- ▷ la droite de régression obtenue à partir des données normalisées,
- ▷ la courbe de convergence illustrant la diminution de l'erreur quadratique moyenne (MSE).

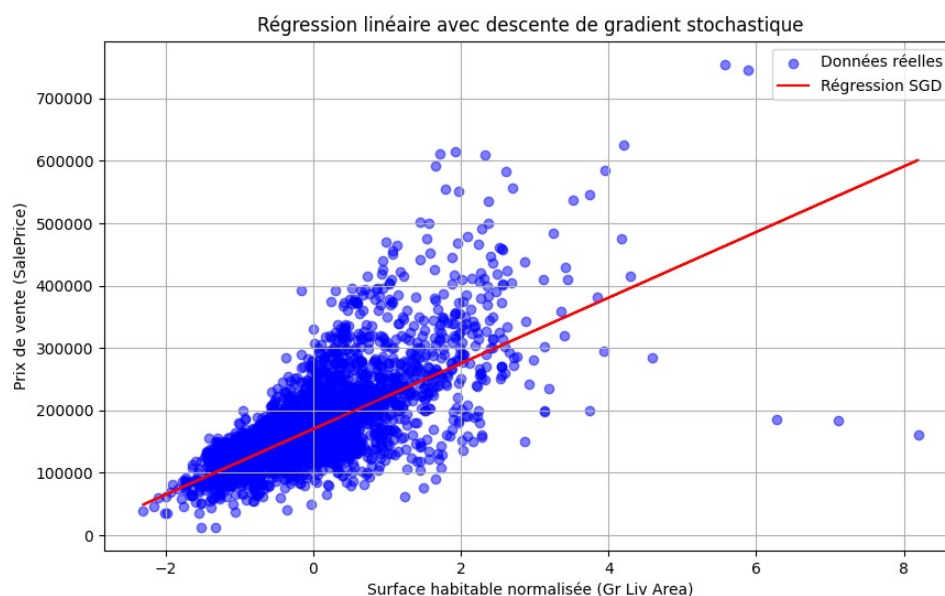


FIGURE 6.4 – Régression linéaire obtenue par descente de gradient stochastique (SGD)

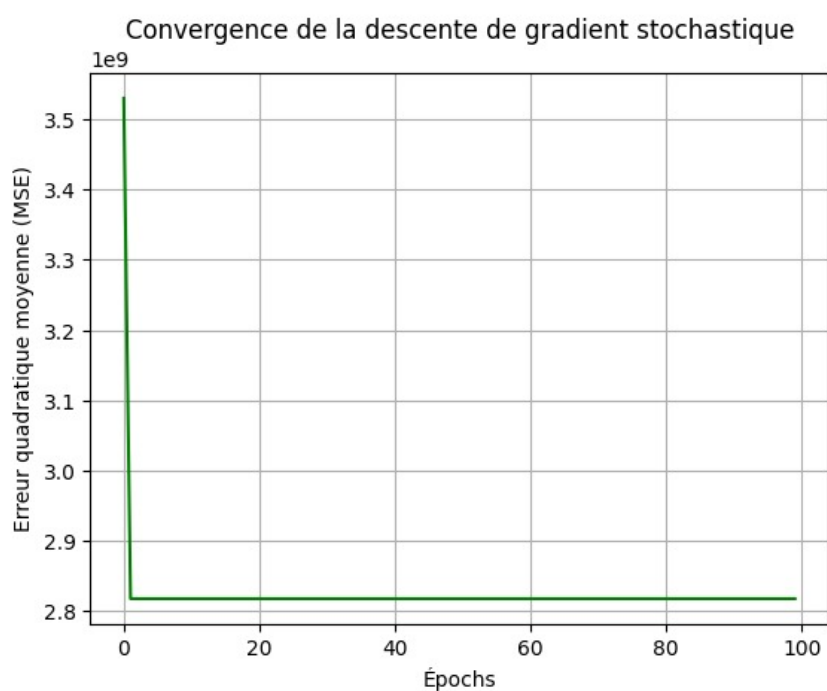


FIGURE 6.5 – Évolution de l'erreur quadratique moyenne (MSE) au fil des époques

Étude de cas : Prédiction du prix des logements par régression linéaire

Données utilisées

Le tableau suivant présente un échantillon de cinq logements, caractérisés par leur surface (en mètres carrés) et leur prix (en milliers de dirhams). Ces données serviront de base pour modéliser la relation linéaire entre la surface et le prix.

Surface (m ²)	Prix (kDH)
50	850
60	900
80	1100
100	1500
120	1700

À partir de cet échantillon, nous allons construire un modèle de régression linéaire, estimer ses paramètres à l'aide des trois méthodes mentionnées, puis analyser les différences obtenues en termes de précision et de performance.

Méthode analytique : Moindres carrés et fonction de coût

La relation entre la surface d'un logement (x) et son prix (y) peut être modélisée par une droite de régression de la forme :

$$y = ax + b$$

où :

- a est le coefficient directeur (la pente de la droite),
- b est l'ordonnée à l'origine (la valeur de y lorsque $x = 0$).

Les paramètres a et b peuvent être calculés directement à l'aide de la méthode des moindres carrés selon les formules suivantes :

$$a = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} \quad \text{et} \quad b = \frac{\sum y_i - a \sum x_i}{n}$$

L'implémentation suivante en Python applique cette méthode à notre jeu de données et affiche la droite de régression correspondante :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x = np.array([50, 60, 80, 100, 120])
4 y = np.array([850, 900, 1100, 1500, 1700])
5 n = len(x)
6 # Calcul des coefficients
7 a = (n * np.sum(x * y) - np.sum(x) * np.sum(y)) / (n * np.sum(x**2) - (np.sum(x))**2)
8 b = (np.sum(y) - a * np.sum(x)) / n
9 # Calcul de la fonction de coût J
```



```

10 J = (1 / (2 * n)) * np.sum((a * x + b - y) ** 2)
11 print(f"Coefficient directeur a = {a:.2f}")
12 print(f"Ordonnée à l'origine b = {b:.2f}")
13 print(f"Valeur de la fonction de coût J = {J:.2f}")
14 # Visualisation
15 plt.figure(figsize=(8, 5))
16 plt.scatter(x, y, color='blue', label="Données observées")
17 plt.plot(x, a * x + b, color='red', label=f"Droite ajustée : y = {a:.2f}x + {b:.2f}")
18 plt.title("Régression linéaire - Méthode des moindres carrés")
19 plt.xlabel("Surface (m²)")
20 plt.ylabel("Prix (kDH)")
21 plt.legend()
22 plt.grid(True)
23 plt.show()

```

Listing 6.7 – Implémentation de la méthode des moindres carrés

Visualisation graphique :

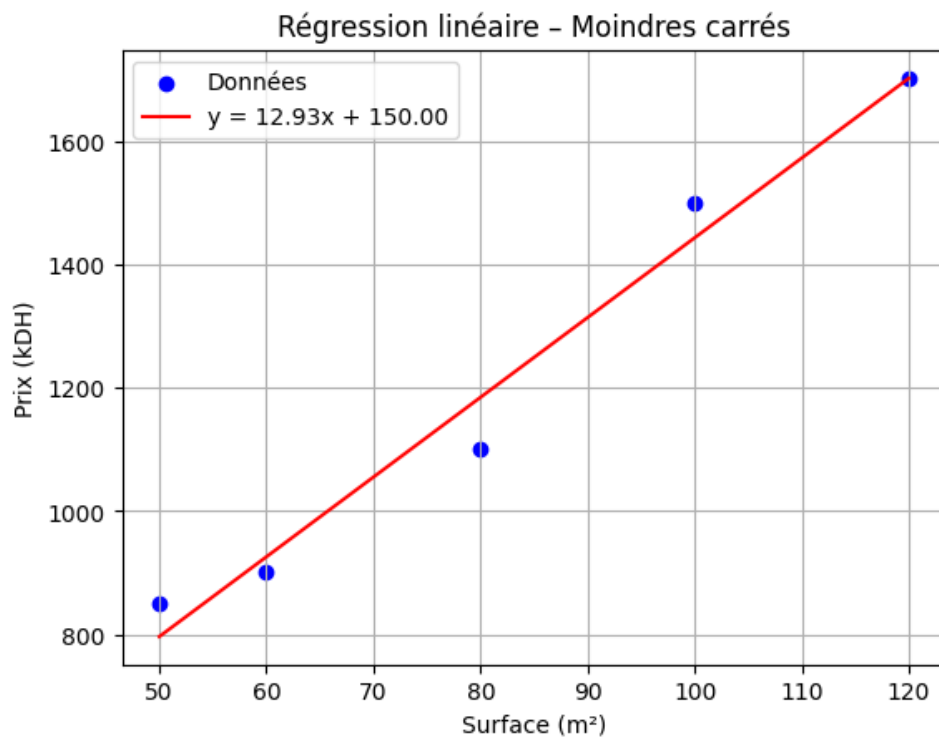


FIGURE 6.6 – représentation graphique

Descente de gradient (GD)

Contrairement à la méthode analytique qui fournit une solution exacte, la **descente de gradient** est une méthode itérative permettant d'ajuster progressivement les paramètres du modèle pour minimiser l'erreur.

À chaque itération, les paramètres a et b sont mis à jour selon les formules suivantes :

$$a \leftarrow a - \alpha \cdot \frac{2}{n} \sum_{i=1}^n (ax_i + b - y_i)x_i$$

$$b \leftarrow b - \alpha \cdot \frac{2}{n} \sum_{i=1}^n (ax_i + b - y_i)$$

où :

- α est le *taux d'apprentissage* (learning rate),
- n est le nombre d'échantillons,
- (x_i, y_i) sont les valeurs de surface et de prix.

Cette méthode permet de suivre l'évolution de l'erreur à chaque itération et d'observer la convergence progressive vers une droite optimale.

L'exemple suivant implémente cette approche en Python et affiche à la fois l'évolution de l'erreur quadratique moyenne et la droite de régression finale.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 # 1. Données d'entr e
4 X = np.array([50, 60, 80, 100, 120])
5 y = np.array([850, 900, 1100, 1500, 1700])
6 n = len(X)
7
8 # 2. Initialisation des param tres
9 m = 0.0
10 b = 0.0
11 # 3. Hyperparam tres
12 learning_rate = 0.00001
13 epochs = 150
14 # 4. Pour suivre l' volution du co t
15 cost_history = []
16
17 # 5. Algorithme de Descente de Gradient Classique (Batch)
18 for epoch in range(epochs):
19     y_pred = m * X + b
20     error = y_pred - y
21
22     # Calcul des gradients (moyenne sur tout le dataset)
23     gradient_m = (2 / n) * np.dot(error, X)
24     gradient_b = (2 / n) * np.sum(error)
25
26     # Mise à jour des param tres
27     m = m - learning_rate * gradient_m
28     b = b - learning_rate * gradient_b
29
30     # Calcul du co t MSE
31     mse = np.mean(error ** 2)
32     cost_history.append(mse)
33
34 # 6. Résultats
35 print("--- Résultats avec Descente de Gradient (GD) ---")
36 print(f"Param tre 'm' final : {m:.4f}")
37 print(f"Param tre 'b' final : {b:.4f}")
38 print(f"Mod le final : Prix = {m:.2f} * Surface + {b:.2f}")

```

```

39
40 # 7. Pr diction exemple
41 surface_test = 75
42 prix_estime = m * surface_test + b
43 print(f"\nPr diction pour une surface de {surface_test} m : {prix_estime:.2f} kDH")
44
45 # 8. Visualisation
46 plt.figure(figsize=(12, 5))
47 # Graphique 1 : Droite de r gression
48 plt.subplot(1, 2, 1)
49 plt.scatter(X, y, color='blue', label='Donn es r elles')
50 plt.plot(X, m * X + b, color='green', label='R gression (GD)')
51 plt.title('R gression Lin aire (GD)')
52 plt.xlabel('Surface (m )')
53 plt.ylabel('Prix (kDH)')
54 plt.legend()
55 plt.grid(True)
56
57 # Graphique 2 : Erreur au fil des poques
58 plt.subplot(1, 2, 2)
59 plt.plot(range(epochs), cost_history, color='green')
60 plt.title(" volution de l'Erreur (MSE) - GD")
61 plt.xlabel(" poques ")
62 plt.ylabel("Erreur Quadratique Moyenne")
63 plt.grid(True)
64 plt.tight_layout()
65 plt.show()

```

Listing 6.8 – Implémentation de la descente de gradient

Visualisation graphique :

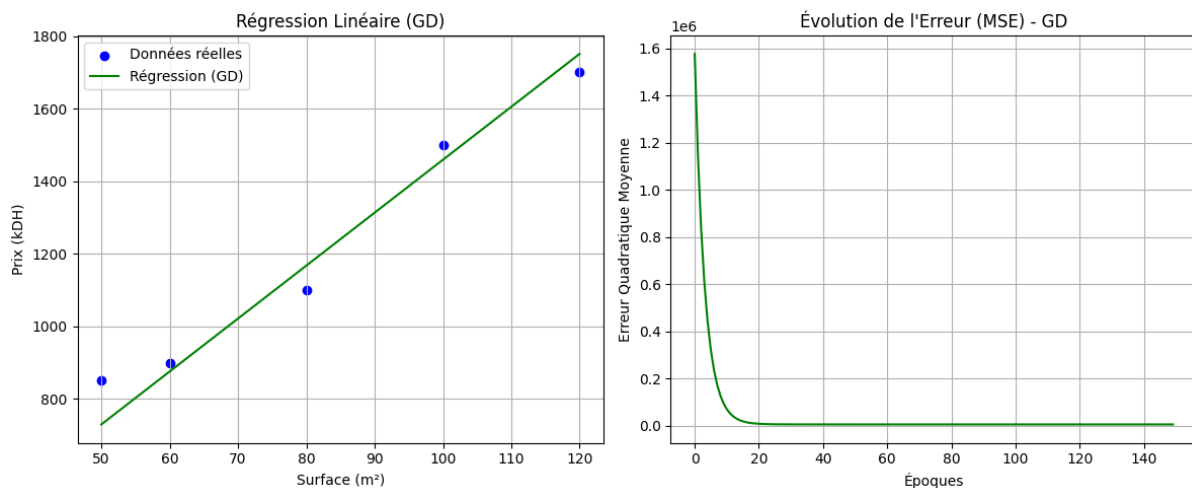


FIGURE 6.7 – representation graphique

Descente de gradient stochastique (SGD)

La **descente de gradient stochastique** met à jour les paramètres après chaque observation, ce qui permet des ajustements rapides et favorise une convergence plus rapide, surtout sur de grands ensembles de données.

Les mises à jour des paramètres a et b à chaque itération sont données par les formules suivantes :

$$\begin{cases} a \leftarrow a - \alpha \cdot 2 \cdot (a \cdot x_i + b - y_i) \cdot x_i \\ b \leftarrow b - \alpha \cdot 2 \cdot (a \cdot x_i + b - y_i) \end{cases}$$

où (x_i, y_i) représente un seul point de données, sélectionné aléatoirement à chaque itération, et α est le taux d'apprentissage.

L'exemple Python ci-dessous illustre cette méthode, en affichant l'évolution de l'erreur quadratique moyenne (MSE) au fil des époques, ainsi que la droite de régression finale.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import random
4 # --- 1. Données et Hyperparamètres ---
5 X = np.array([50, 60, 80, 100, 120])
6 y = np.array([850, 900, 1100, 1500, 1700])
7 n = len(X)
8 # Hyperparamètres
9 learning_rate = 0.00001
10 epochs = 150
11 # --- 2. Initialisation ---
12 m = 0.0
13 b = 0.0
14 cost_history = []
15
16 # --- 3. Algorithme de la Descente de Gradient Stochastique (SGD) ---
17
18 print("D but de l'entraînement avec SGD...")
19
20 # Créer une liste d'indices de 0 à n-1 pour le mélange
21 indices = list(range(n))
22 # Boucle externe pour les époques
23 for epoch in range(epochs):
24
25     # Mélanger les données au début de chaque époque
26     random.shuffle(indices)
27
28     # Boucle interne pour parcourir chaque point de données, un par un
29     for i in indices:
30         # Sélectionner UN SEUL point (x, y)
31         x_i = X[i]
32         y_i = y[i]
33
34         # Prédire la valeur pour ce seul point
35         y_pred = m * x_i + b
36
37         # Calculer l'erreur pour ce seul point
38         error = y_pred - y_i
39
40         # Calculer les gradients (la direction de la correction) pour ce point
41         gradient_m = 2 * error * x_i
42         gradient_b = 2 * error
43
44         # Mettre à jour les paramètres IMMÉDIATEMENT
45         m = m - learning_rate * gradient_m
46         b = b - learning_rate * gradient_b
47

```

```

48     # la fin de chaque poque , on calcule le co t global pour le graphique
49     y_all_pred = m * X + b
50     epoch_cost = np.mean((y_all_pred - y) ** 2)
51     cost_history.append(epoch_cost)
52 print("Entra nement termin .")
53
54 # --- 4. Affichage des R sultats ---
55 print("\n=== R sultats du mod le SGD ===")
56 print(f"Pente 'm' finale : {m:.4f}")
57 print(f"Ordonn e l'origine 'b' finale : {b:.4f}")
58 print(f"Mod le final : Prix {m:.2f} * Surface + {b:.2f}")
59
60 # --- 5. Visualisation des R sultats du SGD ---
61 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
62
63 # Graphique 1 : La Droite de R gression
64 ax1.scatter(X, y, color='black', zorder=5, label='Donn es r elles')
65 ax1.plot(X, m * X + b, color='red', linestyle='--', linewidth=2, label='R gression SGD',
66         )
67 ax1.set_title("1. Mod le de R gression Final (SGD)", fontsize=14)
68 ax1.set_xlabel("Surface (m )")
69 ax1.set_ylabel("Prix (kDH)")
70 ax1.legend()
71 ax1.grid(True)
72
73 # Graphique 2 : volution de l'Erreur (en zigzag)
74 ax2.plot(epochs, cost_history, color='red', alpha=0.9, label='Erreur SGD')
75 ax2.set_title("2. volution de l'Erreur (SGD)", fontsize=14)
76 ax2.set_xlabel(" poques ")
77 ax2.set_ylabel("Erreur Quadratique Moyenne (MSE)")
78 ax2.set_yscale('log') # chelle logarithmique pour bien voir le zigzag
79 ax2.legend()
80 ax2.grid(True, which="both")
81 plt.tight_layout()
82 plt.show()

```

Listing 6.9 – Implémentation de la descente de gradient stochastique (SGD)

Visualisation graphique :

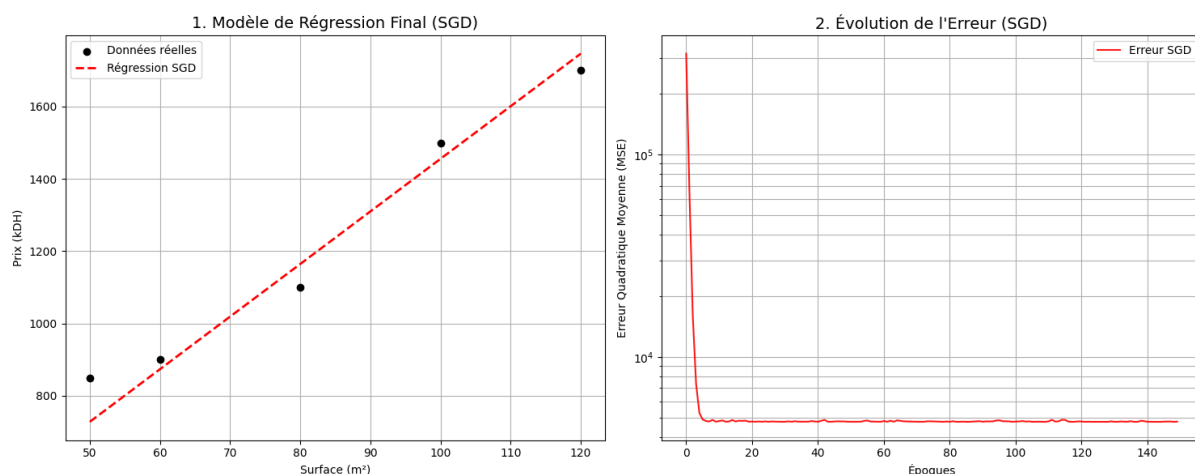


FIGURE 6.8 – représentation graphique

6.2 Classification

6.2.1 Classification d'Emails avec Support Vector Machine (SVM)

• Contexte

Le volume croissant d'emails indésirables (spam) constitue un problème majeur. Ce projet vise à développer un modèle d'apprentissage automatique capable de classer automatiquement les emails en deux catégories : "Spam" ou "Non Spam". Pour ce faire, nous utilisons l'algorithme Support Vector Machine (SVM). L'implémentation est réalisée en Python à l'aide de la bibliothèque scikit-learn. Le dataset utilisé pour ce projet est un fichier simplifié contenant deux caractéristiques (features) pour chaque email : le nombre de fautes d'orthographe et le nombre de mots-clés suspects. L'objectif final est d'entraîner le modèle sur ces données pour qu'il puisse prédire la nature d'un nouvel email inconnu.

• Étape 1 : Importation des Bibliothèques

```
import pandas as pd #Importer les Dataset
import numpy as np #Manipuler le Dataset comme matrice
import matplotlib.pyplot as plt #Visualiser Les données
from sklearn.svm import SVC #Contient l'algorithme de SVC
```

FIGURE 6.9 – Les Bibliothèques

-pandas (alias pd) : C'est la bibliothèque fondamentale pour la manipulation de données en Python. Elle nous permet de charger le fichier CSV dans une structure de données optimisée appelée DataFrame, facilitant ainsi la lecture et la sélection des colonnes.

-numpy (alias np) : La bibliothèque de calcul scientifique par excellence. scikit-learn est construit sur NumPy et requiert que les données d'entrée soient sous forme de tableaux (arrays) NumPy pour les opérations matricielles.

-matplotlib.pyplot (alias plt) : L'outil standard pour la création de graphiques et de visualisations. Il est essentiel pour inspecter les données et, plus tard, pour visualiser la frontière de décision apprise par notre modèle.

-sklearn.svm.SVC : C'est l'importation spécifique de notre algorithme. SVC signifie Support Vector Classifier. C'est la classe qui implémente le modèle SVM pour les tâches de classification.

• Étape 2 : Chargement et Inspection des Données

```
df = pd.read_csv('Dataset/mails.csv', delimiter=";")
df.head()
```

	fautes d'orthographe	nbr des mots-clés déclencheurs	Spam
0	1	0	0
1	2	3	0
2	3	2	0
3	4	1	0
4	5	5	1

FIGURE 6.10 – Chargement et Inspection des Données

- ◇ **pd.read_csv(...)** : Cette fonction de pandas charge les données depuis un fichier CSV.
 'Dataset/mails.csv' est le chemin vers notre fichier de données.
 delimiter=";" est un paramètre crucial ici. Il spécifie que les colonnes dans notre fichier sont séparées par des points-virgules, et non par des virgules (le délimiteur par défaut).
- ◇ **df.head()** : Cette méthode affiche les 5 premières lignes du DataFrame df. C'est une étape de vérification indispensable pour :
 1. Confirmer que le fichier a été chargé correctement.
 2. Comprendre la structure des données : les noms des colonnes (fautes d'orthographe, nbr des mots-clés déclencheurs, Spam) et le type de valeurs qu'elles contiennent.
- ◇ **Sortie** : La table affichée nous montre que nos données sont numériques. La colonne Spam utilise un encodage binaire : 0 pour "Non Spam" et 1 pour "Spam".

• Étape 3 : Prétraitement et Préparation des Données (Feature Engineering)

Avant d'entraîner le modèle, les données brutes doivent être transformées en un format que l'algorithme peut comprendre.

```
X1 = df.iloc[:,0].values #colonne des fautes d'orthographe
X2 = df.iloc[:,1].values #colonne des nbr des mots-clés déclencheurs
Y = df.iloc[:,-1].values #colonne spam ou non spam

X1 = X1.reshape(len(X1),1) #redimensionner X1 vers un vecteur colonne
X2 = X2.reshape(len(X2),1)
X = np.hstack((X1, X2)) #Concatenation des features

Y = Y.reshape(len(Y),1)
```

FIGURE 6.11 – Séparation des Caractéristiques (X) et de la Cible (Y)

◇ **Séparation :**

- `df.iloc[:,0]` sélectionne toutes les lignes de la première colonne (fautes d'orthographe). `df.iloc[:,1]` fait de même pour la deuxième (mots-clés). `df.iloc[:, -1]` sélectionne la dernière colonne (Spam).
- `.values` convertit ces colonnes (qui sont des `pandas.Series`) en tableaux `numpy`.
- `X` représente les caractéristiques (ou variables prédictives).
- `Y` représente la **cible** (ce que nous voulons prédire).

◇ **Mise en Forme (Reshaping) :** `X1.reshape(len(X1), 1)` transforme le tableau `X1` de `(n,)` (un vecteur ligne) en `(n, 1)` (un vecteur colonne). C'est une étape technique nécessaire avant de pouvoir les combiner horizontalement.

◇ **Combinaison des Caractéristiques :** `np.hstack((X1, X2))` (Horizontal Stack) empile les deux vecteurs colonnes `X1` et `X2` côte à côte pour former une matrice `X` unique de dimension `(n-samples, n-features)`, soit `(10, 2)` dans notre cas. C'est le format standard attendu par `scikit-learn`.

```
Y
array([[0],
       [0],
       [0],
       [0],
       [1],
       [1],
       [1],
       [1],
       [1],
       [1]])

X
array([[ 1,  0],
       [ 2,  3],
       [ 3,  2],
       [ 4,  1],
       [ 5,  5],
       [ 6,  7],
       [ 7,  6],
       [ 8,  6],
       [ 9,  7],
       [10,  8]])
```

FIGURE 6.12 – sortie

Sortie : Ces sorties affichent les tableaux `Y` et `X` finaux, confirmant que nos données sont maintenant des matrices `NumPy` prêtes pour l'entraînement.

• Étape 4 : Instanciation et Entraînement du Modèle

```
# fit the model
model = SVC(kernel="linear")
model.fit(X,Y)
```

FIGURE 6.13 – Instanciation et Entraînement du Modèle

- ◇ `model = SVC(kernel="linear")` :
 - On crée une instance de l'objet SVC.
 - Le paramètre `kernel="linear"` est fondamental. Il instruit le SVM de trouver une **frontière de décision linéaire** (une ligne droite dans notre cas 2D) pour séparer les classes. C'est le choix le plus simple, adapté aux problèmes où les données sont supposées être "linéairement séparables".
- ◇ `model.fit(X, Y)` :
 - C'est la phase d'entraînement. La méthode `fit` prend en entrée les caractéristiques `X` et les étiquettes correspondantes `Y`.
 - L'algorithme SVM calcule les paramètres optimaux de la ligne de séparation (ses coefficients et son ordonnée à l'origine) qui maximise la marge entre les points des deux classes.

```
# fit the model
model = SVC(kernel="linear")
model.fit(X,Y)

/home/outman/anaconda3/lib/python3.9/site-
packages/sklearn/utils/validation.py:1300: DataConversionWarning: A column-
vector y was passed when a 1d array was expected. Please change the shape of y
to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)

SVC(kernel='linear')
```

FIGURE 6.14 – Instanciation et Entraînement du Modèle sortie

- **Avertissement** : `DataConversionWarning` : Le message indique que `Y` a été passé comme un vecteur colonne $(n, 1)$ alors qu'un vecteur ligne $(n,)$ était attendu. `scikit-learn` gère cela automatiquement mais nous avertit. C'est une bonne pratique de s'assurer que `Y` a la bonne forme, par exemple en utilisant `Y.ravel()`.

• Étape 5 : Visualisation de la Frontière de Décision

```
plt.xlabel("x1")
plt.ylabel("x2")
plt.plot(X[:, 0], X[:, 1], 'x')

#a1, a2 et b ou bien theta0, theta1, theta2
theta0, theta1, theta2 = model.coef_[0,0], model.coef_[0,1], model.intercept_[0]

plt.plot(X[:, 0], -theta0*X[:, 0]/theta1 - theta2/theta1,
         label = 'Frontière de décision : $G(X) = a_1 \times x_1 + a_2 \times x_2 + b = 0$', color='m')
plt.legend()

<matplotlib.legend.Legend at 0x7fba3cef82b0>
```

FIGURE 6.15 – code graphique

- ◇ Le premier `plt.plot` affiche les données brutes (X) sous forme de points (les 'x' bleus sur le graphique).
- ◇ L'étape clé est le calcul de la frontière de décision. La ligne est définie par l'équation $a_1 \times x_1 + a_2 \times x_2 + b = 0$. Les coefficients a_1 , a_2 (`model.coef`-) et l'intercept b (`model.intercept`-) ont été calculés par `model.fit`.
- ◇ Pour tracer cette ligne, on exprime x_2 en fonction de x_1 : $x_2 = (-a_1/a_2) \times x_1 - (b/a_2)$. C'est cette équation qui est tracée en magenta.

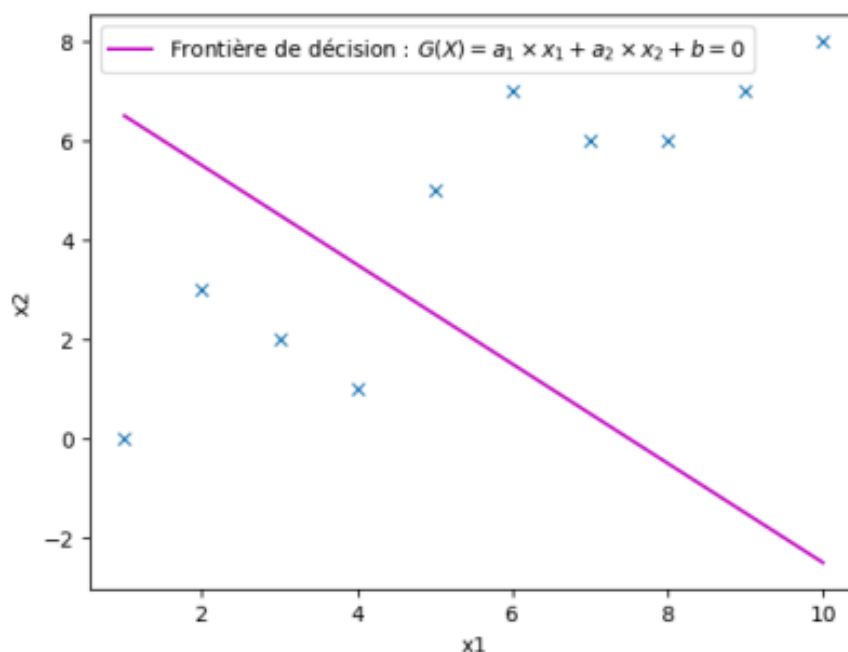


FIGURE 6.16 – représentation graphique

- **Interprétation du graphique :** La ligne magenta est la règle que le modèle a apprise. Tout nouvel email sera projeté sur ce graphique. S'il tombe au-dessus de la ligne, il sera classé comme "Spam" ; s'il tombe en dessous, comme "Non Spam".

• Étape 6 : Évaluation et Prédiction

Une fois le modèle entraîné, nous devons évaluer sa performance et l'utiliser pour faire des prédictions sur de nouvelles données.

```
model.score(X, Y)
1.0
```

FIGURE 6.17 – Coefficient de Détermination (Score)

- ◇ `model.score()` calcule la précision moyenne (accuracy) du modèle sur les données fournies. Elle compare les prédictions du modèle pour X avec les vraies étiquettes Y .

- ◇ Résultat 1.0 : Cela signifie que le modèle a atteint une précision de 100%. Il a correctement classifié tous les exemples du jeu de données d'entraînement. Pour ce dataset simple, cela indique que les données sont parfaitement séparables par une ligne.

Note importante : Dans un contexte réel, un score de 100% sur les données d'entraînement peut être un signe de sur-apprentissage (overfitting). Il est impératif d'évaluer le modèle sur un jeu de données de test distinct pour obtenir une mesure de performance plus réaliste.

- **Prédictions sur de Nouveaux Exemples :**

```
print(model.predict([[6, 6]]))  
[1]  
  
print(model.predict([[1, 1]]))  
[0]
```

FIGURE 6.18 – Prédictions sur de Nouveaux Exemples

- ◇ **Expérience 1 :**

- Scénario : Un email avec 6 fautes et 6 mots-clés.
- Code : `model.predict([[6, 6]])`. Les doubles crochets `[[...]]` sont nécessaires car `predict` attend un ensemble de données (ici, un ensemble d'un seul échantillon).
- Résultat `[1]` : Le modèle prédit que cet email est un Spam, ce qui est cohérent avec sa position sur le graphique (bien au-dessus de la frontière).

- ◇ **Expérience 2 :**

- Scénario : Un email avec 1 faute et 1 mot-clé.
- Code : `model.predict([[1, 1]])`.
- Résultat `[0]` : Le modèle prédit que cet email est un Non Spam, ce qui est également cohérent avec le graphique (en dessous de la frontière).

Ce projet a démontré avec succès la mise en œuvre d'un classificateur SVM pour la détection de spam. En partant d'un jeu de données brut, nous avons suivi un processus complet : préparation des données, entraînement du modèle, évaluation de sa performance et son utilisation pour des prédictions concrètes. Le modèle linéaire a atteint une précision parfaite sur cet ensemble de données, prouvant que les caractéristiques choisies permettent une séparation claire des classes.

Conclusion

Ce projet de fin d'études nous avait permis d'approfondir notre compréhension des fondements de l'apprentissage supervisé, en combinant des approches théoriques, mathématiques et pratiques autour de deux grandes familles d'algorithmes : la régression et la classification.

Nous avons tout d'abord étudié la régression linéaire sous ses différentes formes (simple, multiple, polynomiale), en explorant ses fondements mathématiques à travers la méthode des moindres carrés, les fonctions de coût, ainsi que les algorithmes d'optimisation tels que la descente de gradient (GD) et sa version stochastique (SGD). Parallèlement, l'étude des machines à vecteurs de support (SVM) nous avait permis de comprendre la notion d'hyperplan optimal, la maximisation de la marge, ainsi que l'usage des noyaux dans des cas non linéaires.

La partie pratique s'était appuyée sur l'écosystème Python, notamment les bibliothèques Scikit-learn, Pandas, NumPy et Matplotlib. Nous avons mis en œuvre les modèles étudiés dans un cas concret de prédiction immobilière, en comparant la performance d'approches standards automatisées et de méthodes implémentées manuellement. Cela nous avait permis de mieux saisir les étapes clés d'un projet de machine learning, du traitement des données jusqu'à l'évaluation des résultats.

Au-delà de la mise en œuvre, ce projet nous avait sensibilisés à l'importance du choix des modèles, des hyperparamètres et des techniques de validation. Il nous avait également permis d'identifier les limites et les forces de chaque méthode selon le contexte des données.

Ce travail aurait pu être prolongé de plusieurs manières : en explorant d'autres algorithmes de régression (Ridge, Lasso) et de classification (arbres de décision, réseaux de neurones), en appliquant les modèles à des bases de données plus complexes, ou encore en expérimentant des techniques d'optimisation plus avancées (Adam, RMSprop). De telles extensions auraient renforcé notre capacité à traiter des problématiques variées dans des domaines aussi riches que la santé, la finance ou l'environnement.

En définitive, ce projet avait constitué une expérience complète et formatrice, nous offrant une base solide pour aborder avec rigueur et confiance des défis en science des données et en intelligence artificielle, tant dans un cadre académique que professionnel.